University of Calabria
Department of Mathematics and Computer Science

Master's Degree Course in Computer Science

Master's Thesis

Lazy Declarative Heuristics in Answer Set Programming: Design and Evaluation of a clingo Propagator

Supervisor at the University of Calabria:
Prof. Carmine Dodaro

Supervisors at the University of Potsdam:
Prof. Torsten Schaub
Prof. Javier Romero

Candidate:
Pierpaolo Spadafora

Student ID 263722

Academic Year 2025/2026

Contents

Abstract

In the area of Answer Set Programming (ASP), ground-and-solve systems such as *clingo* require the instantiation of first-order rules before search is initiated. Although declarative domain-specific heuristics offer useful search guidance, dynamic heuristics which are sensitive to the search state at runtime (e.g., dynamic aggregates) experience exponential blow-up when grounding takes place. On the other hand, lazy-grounding approaches offer heuristic evaluation at decision time based on partial assignments but face the problem of high search overhead since the entire search must be conducted in a lazy manner. This thesis attempts to explore the possibility of incorporating the potential of lazy heuristic evaluation in the ground-and-solve paradigm without compromising its efficiency. We describe a lazy declarative heuristic framework for *clingo* using its propagator interface.

Two execution backends are provided: a compiled C++ one with an incrementally aggregate state and an embedded SWI-Prolog engine with full relational evaluation based on the synchronized trail. The methodology allows decoupling heuristic representation (ground and lazy) from interpretation semantics (clingo-like and Alpha-like) thanks to an order-preserving weight flattening and the appropriate interpretation of the default negation on unassigned atoms. The findings show that this is necessary for triggering dynamic heuristics to ensure proper behavior. Benchmark results on three hard combinatorial optimization problems (Balanced Sum, Partner Units, and House Reconfiguration) show how the lazy propagator avoids combinatorially expensive ground instructions and reduces memory requirements while keeping the optimal search guidance. Compared to the lazy-grounding solver Alpha, our framework shows how the gap between ground-and-solve and lazy systems is closed on dynamic heuristics, bridging a possible gap between static ground solving and lazy solving.

Chapter 1

Introduction and Background

1.1 The Cost of a Dynamic Heuristic

Answer Set Programming (ASP) is a declarative approach to combinatorial search. Instead of describing an algorithm step by step, the modeller describes the properties that an admissible solution must satisfy, and a solver enumerates the assignments that satisfy them [1, 2, 3].

ASP solvers are essentially highly optimized backtracking, or more appropriately *backjumping*, engines. The current state of the art (SOTA) is dominated by two main paradigms: ground-and-solve and lazy grounding.

In ground-and-solve systems, such as clingo, first-order rules are instantiated over the problem domain before search starts, including the domain-specific heuristics that direct the solver's search. This approach is inherently memory-bound, with bottlenecks manifesting in two main scenarios: as problem instances scale, and whenever domain-specific heuristics depend on dynamic search state, for instance through an aggregate over the atoms that are true *at that point of the search*. Since grounding happens before the search starts, its only option is to enumerate every value the aggregate could take, and a single heuristic ends up costing a number of ground objects that is combinatorial in the size of the instance. In contrast, lazy-grounding systems interleave grounding and solving. While this yields significantly higher memory efficiency, allowing a further reach on memory-intensive problems, it comes at the cost of a significantly slower search.

Depending on its underlying architecture, an ASP solver can be seen as an engine that compiles or interprets a very convenient and elegant declarative language [4] to search for a solution. The convenience of that language is paid for in control: the modeller states *what* an admissible solution is, but not *how* the engine should look for one. A pure backtracking approach behaves as an uninformed search, and on problems whose structure is known by construction, guiding the solver with that knowledge is extremely helpful.

Domain-specific heuristics are the means by which the modeller feeds that knowledge into the search. Syntactically they are declarative annotations that the grounder instantiates like ordinary rules, and their purpose is to tell the solver which atom to decide next, and with which polarity; their impact on solving is well documented [5, 6]. They are, however, part of the program, and clingo instantiates

the program before search starts. A heuristic is therefore expanded, in full, before the first decision is taken. That expansion can be relatively harmless as long as the heuristic depends only on the input. It stops being harmless as soon as the heuristic depends on *how the search is going*, which is precisely when a heuristic is most useful. Consider the directive used throughout this thesis to guide Balanced Sum Problem, the problem of splitting $\{1, \ldots, n\}$ into two sets `b/1` and `c/1` of nearly equal sum:

```
1 #heuristic b(X) : x(X), not c(X), S = #sum{Y : c(Y)}, W = ((n+1)*S)+X. [W,
  ↪  true]
```

The rule reads: prefer to place the element X into b, the more strongly the larger the sum S of what is already in c.
The aggregate value S appears *inside the weight*, and the atoms `c(Y)` it sums over are guessed by the search rather than given as facts. The grounder cannot know what S will be. Its only option is to enumerate every value the sum could reach and to materialise a separate ground directive for every pair (X, S) producing $\Theta(n^3)$ ground objects, more than a million at $n = 100$. All of them instantiate the same single rule the modeller wrote, and all but the one that matches the current sum are inert at any given decision. A domain-specific heuristic introduced to help the engine solve a problem quicker ends up putting the answer out of reach.

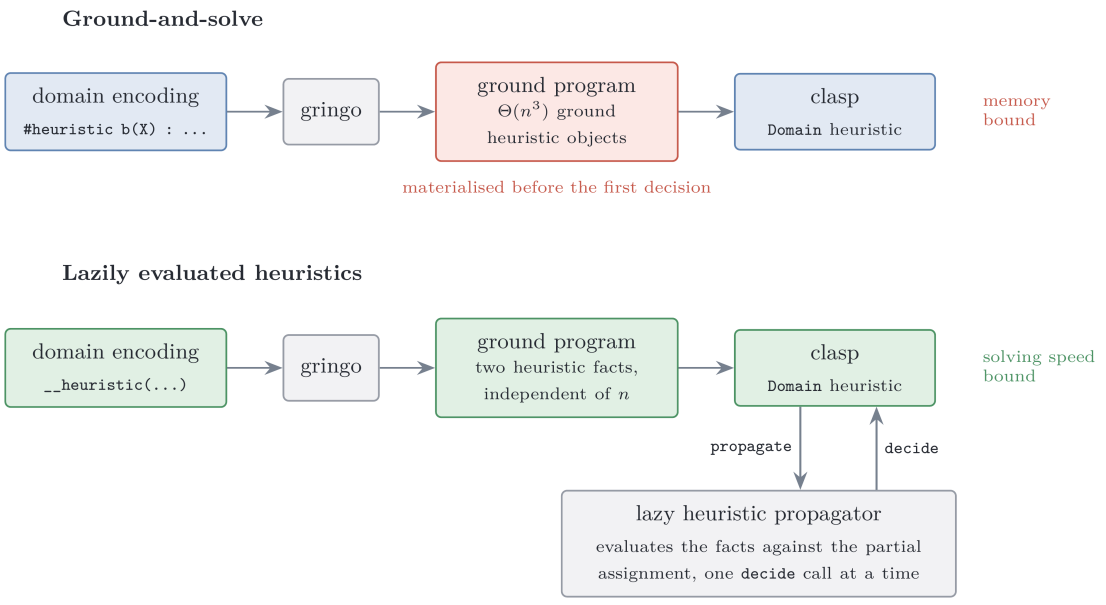

Figure 1.1: The same heuristic under the two approaches compared in this thesis.

The alternative is to leave the heuristic symbolic and to evaluate it while the search runs, when the partial assignment that S refers to actually exists. This is the natural approach in lazy-grounding solvers and it has already been tested [7, 8, 9].

1.2 Answer Set Programming

A *normal rule* in ASP has the form

$$a \leftarrow b_1, \ldots, b_m,\ not\ c_1, \ldots, not\ c_n,$$

where a, the b_i and the c_j are atoms: the head a is derived whenever every positive body atom b_i is derivable and no negative body atom c_j is. The negation *not* is *default negation*: *not* c holds in the absence of a derivation for c. Because a rule read this way can justify itself, the meaning of a program is not given by the rules alone but through the notion of a stable model. Fix a set of atoms X, the candidate solution, and a ground program P. The *reduct* P^X is obtained by deleting every rule whose negative body intersects X, and by deleting the negative literals from the rules that survive. The reduct is a positive program, so it has a unique least model, and X is a *stable model* (or *answer set*) of P exactly when it coincides with that least model [1]. On the two-rule program

$$\begin{Bmatrix} a \leftarrow not\ b \\ b \leftarrow not\ a \end{Bmatrix}$$

the candidate $X = \{a\}$ gives the reduct $\{a \leftarrow\}$, whose least model is $\{a\}$: stable. The candidate $\{a, b\}$ deletes both rules, leaving the least model $\emptyset$: not stable. A stable model is then a set of atoms both closed under the rules and justified by them, in which every atom has a non-circular derivation that survives the model's own assumptions about what is false.

In the *guess-and-check* idiom a choice construct opens the solution space, integrity constraints prune the candidates that violate the specification, and the answer sets of the program correspond one-to-one to the solutions of the problem. ASP syntax extends normal rules with first-order variables, arithmetic, conditional literals, and three constructs this thesis uses throughout. A *choice rule* such as `{ b(X) } :- x(X)` leaves the truth of its head open, so its head atoms, called *choice atoms* below, are exactly the ones the solver has to decide. An *integrity constraint* `:- Body.` is a rule with empty head and forbids every model that satisfies *Body*. An *aggregate atom* such as `#sum`, `#count`, `#min` or `#max` condenses a set of weighted contributions into a single comparable value [3]. Aggregates are central to this work: the heuristics under study rank decisions by expressions such as "the current sum of the elements already placed in a set", and how and *when* such an aggregate is evaluated is exactly what differentiates the systems compared here.

1.3 Ground-and-Solve: gringo, clasp, and Conflict-Driven Search

Clingo [10] couples the grounder Gringo with the solver Clasp into a single system. The grounder instantiates the first-order program over its Herbrand universe, producing a variable-free program. To mitigate combinatorial explosion, Gringo restricts this expansion to rule instances whose positive body is potentially derivable;

as a result, the ground program is generally much smaller than a full instantiation, though it must still be materialised in full before search starts. The solver Clasp [11] then searches for the stable models of the ground program using *conflict-driven nogood learning* (CDNL), an algorithm that learns from conflict-inducing choices. The solver maintains a *partial assignment*, a consistent set of literals over the program atoms; it alternates *unit propagation* steps with *decision* step, and when propagation derives a contradiction, it analyses the conflict, learns a nogood built around the *First Unique Implication Point*(1UIP), and *backjumps* to the decision level at which that nogood becomes unit. Which atom to decide on, and with which polarity, is delegated to a *decision heuristic*; clasp's default is activity based, in the family introduced by [12], and favours atoms that appear in recent conflicts.

This approach entails that every atom and every rule clasp will ever see must exist before the first decision is made and also that the size of the ground program is the dominant memory cost of the pipeline. *Grounding bottleneck* is a well-known limitation of ground-and-solve systems [8]; Chapter 4 measures one concrete manifestation of it, namely heuristic directives whose weight depends on an aggregate value.

1.4 Domain-Specific Heuristics in clingo

Clasp allows the modeller to steer its decision heuristic using domain-specific knowledge through the `#heuristic` directive [5]:

```
#heuristic A : Body. [W@P, M]
```

Here, A is the target domain atom whose decision choice is steered in the direction specified by the *modifier* M. This heuristic applies only if the body of the heuristic rule is satisfied. Among the grounded heurisics the one with a greater weight W has precedence; if two heuristics have the same target atom then the one with the higher priority P will be chosen for the comparison

1.5 Lazy Grounding, Alpha, and Heuristics over Partial Assignments

Alpha [7] is the reference SOTA lazy-ground solver for this work.

Comploi-Taupe et al. extended Alpha with declaratively specified domain-specific heuristics evaluated against the *partial* assignment maintained during solving [8]. Its semantics differs from clingo's on two main points.

Specifically, Alpha evaluates negated body literals against unassigned atoms and prioritizes P over weight W in the tuple $(W@P)$, always preferring higher priority values. Only when two atom, even different, have the same priority are they compared by weight. A complete formalization of these semantic differences and their integration into our experimental design is provided in Chapter 2.

1.6 Extending clingo: Propagators and the Heuristic Interface

Nothing in the previous sections suggests that clingo can be made to evaluate anything at search time. It does, however, expose a supported way in: *propagators*, user-supplied components registered with the solver that take part in the search loop [10]. This thesis uses that door to add *guidance* that is computed rather than materialised.

A propagator has four callbacks. During `init`, it inspects the symbolic atoms of the ground program, maps the atoms it cares about to solver literals, and registers *watches* on them. During search, `propagate` is invoked when a watched literal becomes true, `undo` when backtracking retracts it, and `check` on total assignments. The propagator is woken only by clasp when a change occurs in an atom it asked to observe.

The `Clingo::Heuristic` interface inherits from Propagator and extends it with a `decide` callback, invoked at each decision step *before* the solver's built-in heuristic. The Heuristic object receives the current assignment of the solver and either returns a signed literal, which becomes the next decision, or returns 0, in which case Clasp falls back to its default activity-based heuristic (VSIDS [12]). This is the mechanism the rest of the thesis builds on. It keeps the declarative heuristic descriptions in memory, evaluates them incrementally against the trail, and answers each `decide` call with the best applicable target.

1.7 Objective and Contributions

This work asks whether the same effect of lazily evaluating heuristics can be obtained inside clingo without giving up its ground-and-solve pipeline and speed advantage, that is, whether a ground-and-solve engine can be equipped with lazily evaluated heuristics (Figure 1.1) and if the overhead of this can still allow us to outperform a **SOTA** lazy-ground solver.

The thesis makes the following contributions.

1. **A lazy heuristic evaluator for clingo.** A propagator, registered through the public `Clingo::Heuristic` interface, that keeps a declarative heuristic in memory as a compact description and evaluates it at every decision against the current partial assignment.
2. **Two backends** A compiled C++ backend, which materialises one candidate per ground target and maintains incremental aggregate states, and an in-process SWI-Prolog backend, which evaluates the heuristic as an ordinary logic program against a mirrored image of the solver trail.
3. **Separating representation from semantics.** Ground-and-Solve, Lazy, Clingo semantics and Alpha semantics combined give four variants of every encoding plus a heuristic-free baseline, so that no measured difference has to be attributed to two causes at once.

4. **Flattening of the weights for Alpha-style heuristics.** Alpha's priority levels are global, while clingo's are local to a single atom. Section 2.1.2 presents the transformation that folds a level into the weight, the conditions under which it preserves the intended order and the weight field limit imposed by clingo.

5. **Trade-off analysis of the lazy approach.** By collecting per-decision metrics in both backends, we evaluate the exact computational overhead of lazy grounding against its savings in program size.

The evaluation covers three problem families, Balanced Sum Problem, the Partner Units Problem and the House Reconfiguration Problem, on both backends, with a correctness harness that checks that no variant changes the answer sets. The implementation is available at [1]

[1] `https://github.com/pierspad/clingo-lazy-heuristic-evaluator`

Chapter 2

Methodology

2.1 The Experimental Design: Two Axes, Four Variants

Every experiment in this thesis compares encodings of the same problem that differ in how the domain-specific heuristic is represented and interpreted. The rules that define the problem, its guess and its constraints, are held fixed across the variants of a family; what varies are the two properties that Section 1.4 and Section 1.5 identified as the points on which clingo and Alpha disagree, and they vary independently.

The *approach* axis fixes where the heuristic is evaluated. In the ground-and-solve approach the heuristic is written with native `#heuristic` directives: gringo expands them before search and clasp's Domain heuristic consumes the resulting ground objects. In the lazy approach the heuristic remains a handful of ground facts, `__heuristic(...)` for the native backend and `heuristic("...")` for the Prolog one, which the propagator reads at its initialisation and evaluates at every decision step against the partial assignment.

The *semantics* axis fixes how a heuristic condition reads partial information: the clingo-like semantics, under which `not a` holds only once a has been assigned false and an aggregate has a value only once its sources are determined, against the Alpha-like semantics, under which `not a` holds as soon as a is not assigned true, and an aggregate is computed over the atoms currently true. Only the second reading makes a heuristic dynamic in the sense of reacting to a partial assignment during search, as motivated in 1.1: An instruction such as "prefer the element whose insertion keeps the two sums balanced" presupposes that the sum can be evaluated against an assignment that is still being built; under the clingo's semantics, that sum simply has no value until it is too late for the guidance to matter.

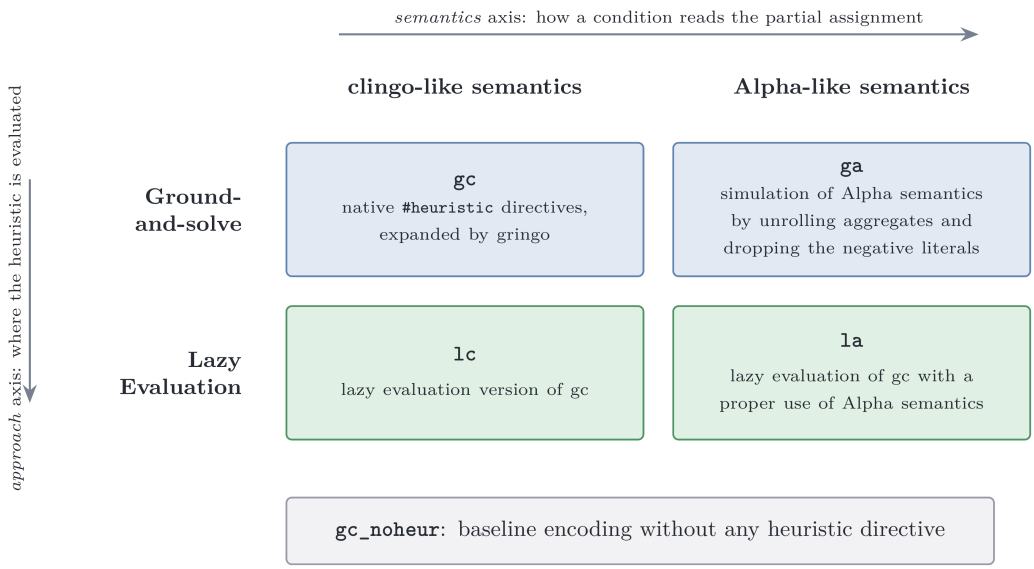

Figure 2.1: The experimental matrix. Prefixes name the approach, suffixes the semantics.

Crossing the two axes yields the four variants that the rest of the thesis refers to by: `gc` and `ga` for the ground approach under the clingo and the Alpha semantics, `lc` and `la` for the lazy evaluated one, the prefix naming the approach and the suffix the semantics. To them is added the heuristic-free baseline `gc_noheur`, which carries the same domain logic with no heuristic at all and measures what the solver does when it is left alone (Figure 2.1).

Ideally, changing the approach or semantics should modify only the heuristic representation. On BSP, this control is exact, as the lazy encodings simply add two facts to the baseline program. On PUP and HRP, however, lazy encodings require auxiliary predicates to expose aggregate sources and align negative conditions, yielding a ground program measurably larger than the baseline (Sections 3.3 and 4.11). Nevertheless, when comparing the lazy variants among themselves, such as `la` against `lc` across all three families, these auxiliary predicates are shared by both encodings, the difference disappears and variable isolation remains exact by construction.

2.1.1 Simulating Alpha in Ground clingo

The `ga` encodings apply two systematic transformations to `gc`.

The first transformation drops negative literals that test the partial state (such as `not c(X)` in BSP, `not not_assigned_...` in PUP, and guards like `not fullCabinet(C)` in HRP). Under Alpha-like semantics, these literals are satisfied by unassigned atoms as well as false ones. Since a target atom is unassigned at the moment a decision is evaluated, dropping the negative literal altogether provides the closest approximation in ground clingo; this is harmless in practice because the solver never re-decides an atom that is already assigned.

The second transformation replaces the equality condition with a *lower/upper* bound. For BSP:

```
sum_value(0..tot).
#heuristic b(X) : x(X), sum_value(S), S <= #sum{Y : c(Y)},
    W = ((n+1)*S)+X. [W, true]
```

Instead of binding S to the final value of the sum, the directive is unrolled over all candidate values `sum_value(S)` and guarded by the condition $S \leq \#\mathtt{sum}\{Y : c(Y)\}$. During search, as soon as the partial sum reaches a bound S, every directive with that bound becomes applicable; among the applicable ground directives for the same atom, clasp's Domain heuristic retains the one with the maximum weight, which corresponds exactly to the *current* value of the aggregate. The max-weight selection thus reconstructs the dynamic aggregate at solving time. For aggregates whose contribution to the weight decreases rather than increases (such as the free-capacity terms N_0, N_1, N_2 in PUP) the same transformation is applied with the bound $V \geq \#\mathtt{max}\{\ldots\}$, so that the max-weight selection picks the smallest proven value.

This simulation is faithful, but its price is exactly the combinatorial explosion that motivated the thesis: one ground directive per point of the product of the unrolled domains, measured in Chapter 4.

2.1.2 Weights, Priorities, and Flattening

Section 1.5 recorded that the two systems read the annotation $[W@P]$ of a heuristic differently. In Alpha, the pair is an *absolute* rank over all heuristic instances, read lexicographically as (P, W) : the priority P is a global level and decides first, so no instance of level P is considered while an instance of a level $P' > P$ is still applicable, and the weight W only orders the instances that share the same level. In clingo, the evaluation is different: P is local, arbitrating only between heuristics relative to the *same* target atom, while the order in which distinct atoms are decided comes from the weight alone. An encoding written for Alpha therefore cannot be handed to clingo verbatim.

The intended order can nevertheless be recovered, by *flattening* the level into the weight:

$$W_{\mathit{flat}} \;=\; W + P \cdot M, \qquad M > \max W - \min W,$$

where the intra-level weights W range over $[\min W, \max W]$ and the level multiplier M is larger than that range. Under this condition $W + P \cdot M > W' + P' \cdot M$ holds whenever $P > P'$, whatever the two weights are, and reduces to $W > W'$ when $P = P'$: the total order induced on the atoms is then the one Alpha's absolute levels induce. When all weights are non-negative, which is the case in every encoding of this suite, $M > \max W$ suffices.

Each benchmark fixes its own M. In the PUP encodings M is the constant offset 100,000 added to the zone weights, against a maximum sensor weight of 26,220 on the benchmark instances; in HRP M is derived from the instance as `levelMul(LM) :- LM = MC + MR + 10` over the maximum cabinet and room identifiers,

which gives 16 on the smallest instance and 54 on the largest. BSP doesn't need a multiplier, as discussed in Section 3.3.1.

The rule has one further condition, which is not a property of the theoretical order but of the machine that consumes it. Clasp stores the weight of a `#heuristic` modification in a signed 16-bit integer field and clamps any value exceeding 32,767. Consequently, two ground directives whose weights both exceed 32,767 become indistinguishable to Clasp's Domain heuristic. A flattened weight must therefore satisfy $W + P \cdot M \leq 32{,}767$ for the ground variants to rank as intended. HRP stays far below this bound; PUP exceeds it on the zone directives, with the consequences discussed in Section 4.11; and BSP, whose maximum weight grows as $\Theta(n^3)$, crosses it within the benchmarked range, as analyzed in detail in Section 4.8. The lazy backends are not subject to this bound because they rank targets internally and hand Clasp a ready decision rather than a weight.

The suite adopts clingo's convention in *all four variants and in both backends*: absolute levels always live in the weight, simplifying the propagator as it does not need to account for a separate priority field. Once global levels are folded into weights, weight becomes the sole ordering criterion under both semantics. Accordingly, the native lazy language (Section 2.3) omits the priority field entirely.

A last convention concerns empty aggregates: `#max` over an empty set is 0 in Alpha but *#inf* in clingo. All encodings therefore write `#max{0; ...}` with an explicit neutral element, and both lazy backends implement the same rule (the native `MaxState` returns 0 when empty; the Prolog runtime defines `dyn_max` with a 0 default).

2.2 System Architecture

The project maintains two modified copies of clingo. They share the same upstream sources and differ only in their extensions: the build in `clingo-native` carries the C++ backend, while the build in `clingo-prolog` carries the Prolog backend, configured with `CLINGO_USE_SWIPL=ON` so that the SWI-Prolog engine is linked in-process, making its memory impact easier to account for. In both builds, the extension lives entirely under `libclingo`, exposes the same class name, and is registered at the same point of `clingo_app.cc`:

```
1 HeuristicPropagator my_propagator;
2 ctl.register_propagator(my_propagator, true);
```

The propagator is registered on *every* run of the modified binary. On a program that contains no heuristic facts, it remains inert because `init` finds nothing to watch and `decide` always returns 0; the same binary therefore runs the ground variants and the heuristic-free baseline without requiring a separate build. The second argument, `true`, instructs Clasp to call the propagator sequentially, avoiding race conditions on the incremental state described in Section 3.1.1. No other parts of Clingo are modified. Figure 2.2 shows the architecture of the two binary build variants.

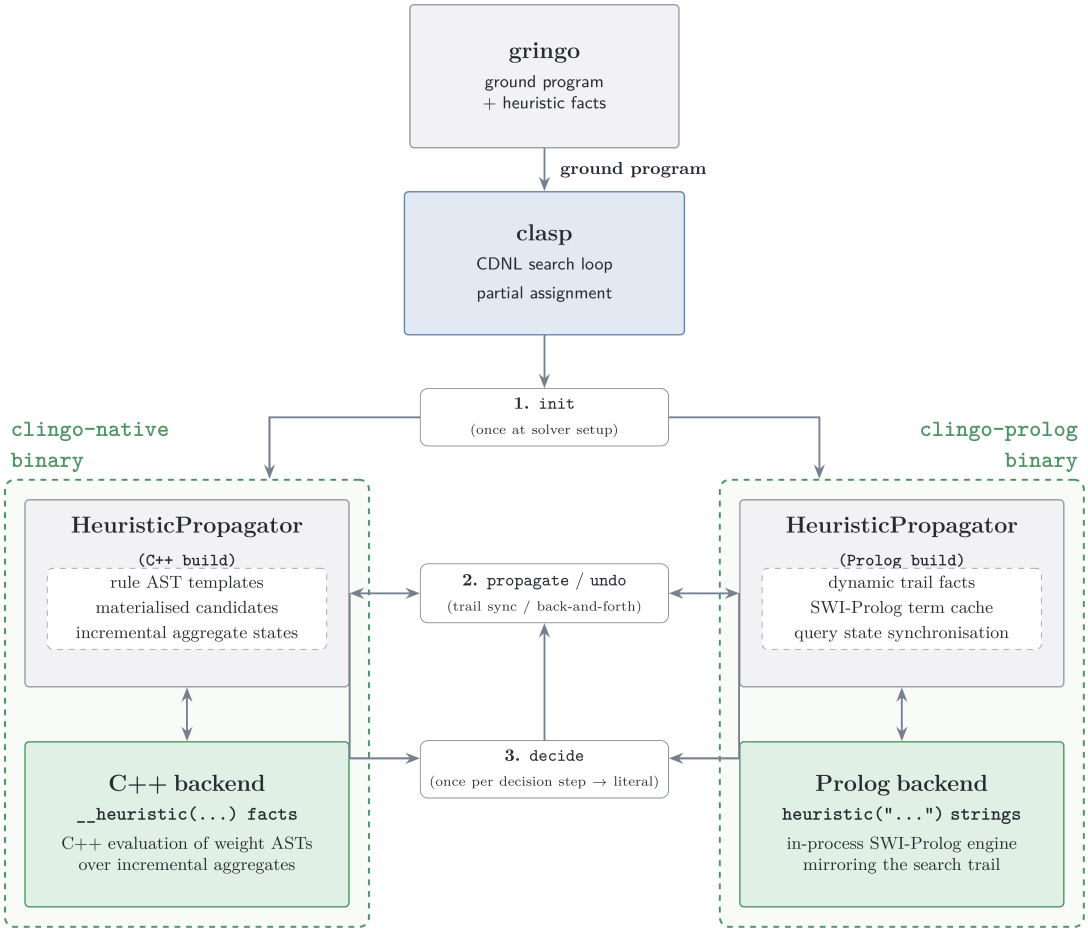

Figure 2.2: System architecture of the two binary variants, showing their respective heuristic propagators and evaluation backends attached to clasp.

In both builds, the class implements `Clingo::Heuristic`—the propagator interface extended with the `decide` callback (Section 1.6). Both backends follow the same four-step lifecycle:

- During `init`, they scan symbolic atoms for trigger facts (`__heuristic(...)` in the C++ build and `heuristic("...")` in the Prolog build), construct their runtime representation, and register watches on solver literals.
- During search, `propagate` and `undo` keep the internal state synchronized with the trail.
- At each decision point, `decide` either returns a signed literal for the highest-ranked target or returns 0, delegating the decision to Clasp's fallback heuristic.

To activate this mechanism, runs are launched with the flag `--heuristic=Domain`. More on the differences in Chapter 3.

The propagator never adds a constraint, never derives a literal and never refuses one. It only reorders the decisions clasp would have taken anyway, so the answer sets are untouched, and a heuristic can make a run slower or faster but cannot change the meaning of the encoding. This is expanded in Section 3.4.

2.3 Translation of the Heuristic Directives

Neither backend reads `#heuristic` directives. A lazy language is being used replaces them with a single ground fact that *describes* them. This section presents what one may write, and what each construct means, by deriving that fact for the BSP heuristic of Section 1.1 one component at a time. How the description is executed is left to Section 3.

The starting point is the directive as clingo reads it:

```
#heuristic b(X) : x(X), not c(X), S = #sum{Y : c(Y)}, W = ((n+1)*S)+X. [W,
↪  true]
```

Target. The target atom is `b(X)`, where X ranges over the ground instances. The lazy fact specifies only the predicate name, `__target(b)`. During `init`, the propagator queries clingo's atom table for all ground instances of `b(...)` already materialized by Gringo.

Positive body. The condition `x(X)` shares its argument with the target: the candidate for `b(7)` is alive only while `x(7)` is true. A condition of this shape is written as the bare predicate name, `x`, and the propagator matches its arguments against the target tuple position by position.

However a body predicate may have a different arity; its arguments may sit in different positions; or a value may have to be read out of it and used in the weight. The correspondence is then stated explicitly:

```
__body(pred, __match(src, tgt), __bind_arg(var, idx))
```

`__match(src, tgt)` requires the argument the `src` position of the body atom to equal the argument at the `tgt` position of the target atom, where both `src` and `tgt` are integers. All indices are 0-based, and at least one `__match` is mandatory. `__bind_arg(var, idx)` extracts argument `idx` of the body atom into a variable.

Given the ground facts:

```
score(1,10). score(2,5). score(3,7).
```

and the target atoms:

```
choose(1). choose(2). choose(3).
```

the directive:

```
__heuristic(__target(choose),
            __body(score, __match(0,0), __bind_arg(w,1)),
            __weight(w), __modifier(true)).
```

pairs each `choose(X)` with the `score(X,W)` that agrees with it on the first argument, and binds `w` to the second argument of that atom. The weights are then 10, 5 and 7, so the propagator decides `choose(1)` first.

Negative body. The condition `not c(X)` likewise matches its arguments against the target tuple. All the parts of the negative body is specified by prefixing the predicate name with `__n_`.

Semantics. The reading of that negation is fixed by the `__semantics(S)` selector, which decides whether `__n_c` means "`c(X)` is assigned false" or "`c(X)` is not assigned true". The same selector fixes the reading of aggregates, described below. It takes `alpha` or `clingo`, and defaults to `clingo`, so that a directive written without it behaves as the native `#heuristic` it replaces.

Aggregates are evaluated lazily during search. `__bind(s, __sum(c))` binds variable `s` to the sum over the predicate `c` (by default aggregating its first argument). The operators `__count`, `__min`, and `__max` follow the same structure.

An aggregate can be restricted *per target* using `__filter(src_idx, tgt_idx, offset)`, which keeps only source atoms whose argument at position `src_idx` equals argument `tgt_idx` of the target plus an optional `offset`. For instance, in PUP, retrieving the maximum number of sensors on the *previous* unit $(U - 1)$ for a target `assigned_zone_unit(Z,U)` is expressed as:

```
__bind(m1, __max(num_sensors_on_unit, 0, __filter(1, 1, -1)))
```

where `0` selects the aggregated argument N in `num_sensors_on_unit(N, U')`, and the filter matches $U' = U - 1$.

Weight is expressed as a *term tree* such as:

```
__weight(__add(__mul(s, B), self))
```

which represents $(s \cdot B + self)$, where `self` is a shorthand for the target atom's argument at index 0 without requiring a variable binding.

The binary constructors `__add`, `__sub`, and `__mul` form expression trees whose leaves belong to three categories: integer constants; variables of the heuristic language and target arguments (accessed through `self` or `__bind_target_arg(var, idx)` for any other).

At `init` the propagator compiles this tree structure into an AST that gets evaluated during search for each active candidate from the current trail, without ever having to reserve extra memory. Weights are always kept positive to avoid them suggesting to not choose an atom

At `init`, the propagator compiles all the term trees into internal ASTs. During search, active RuntimeHeuristicCandidates evaluate their corresponding AST based on the current trail state, avoiding in this way the memory cost of the grounding. Weights must always have positive weights.

At `init`, the propagator compiles all weight term trees into ASTs. During search, each active heuristic object evaluates its weight AST dynamically against the current trail state, avoiding the memory overhead of materializing them in the ground program. Only candidates with a strictly positive resolved weight are taken into account; the others are ignored, leaving their decision to the solver's default heuristic.

Modifier. The `__modifier(M)` selector specifies which clingo's heuristic modifiers to use: `level` sets the ranking value alone, `sign` sets the preferred polarity alone, while `true` (the default) and `false` set both.

The solver's `decide` interface expects a *signed* literal, so a ranking value alone does not determine a decision. Each target keeps one slot per channel, ranking value and preferred polarity, and when several directives write to the same slot the one with the highest weight wins. This is what replaces clingo's local priority. Across the benchmarks all directives use `true`, except for the final layer of HRP, which uses `false` to prune the remaining choice atoms.

Assembling the components gives the fact in full:

```
__heuristic(__target(b), x, __n_c, __bind(s, __sum(c)),
            __weight(__add(__mul(s, B), self)),
            __semantics(alpha), __modifier(true)) :- B = n + 1.
```

construct	clingo directive	native lazy fact	Prolog rule string
structure			
target	`b(X)`	`__target(b)`	`heuristic(b(X), W, P, M)`
positive body, target tuple	`x(X)`	`x`	`holds(x(X))`
positive body, join	`p(X,V)`	`__body(p, __match(0,0), __bind_arg(v,1))`	`holds(p(X,V))`
negation	`not c(X)`	`__n_c`	`alpha_not(c(X))` `clingo_not(c(X))`
aggregate	`S = #sum{Y : c(Y)}`	`__bind(s, __sum(c))` also `__count`, `__min`, `__max`	`aggregate_all( sum(Y), holds(c(Y)), S)`
filtered aggregate	`M = #max{N : q(N,U-1)}`	`__bind(m, __max(q, 0, __filter(1,1,-1)))`	`U1 is U-1, dyn_max( holds(q(N,U1)), N, M)`
target arguments	`X`, `Y` in `b(X,Y)`	`self`, `__bind_target_arg( y, 1)`	ordinary head variables
annotations			
weight	`W = ((n+1)*S)+X`	`__weight(__add( __mul(s,B), self))`	`W is B*S+X`
modifier	`[W, true]`	`__modifier(true)`, default `true`	fourth head argument
semantics	— (fixed by the system)	`__semantics(alpha)` `__semantics(clingo)`	choice of `alpha_not` / `clingo_not`
target still free	— (implicit in clasp)	implicit: an assigned target is skipped	`target_available( b(X))`

Table 2.1: The same heuristic in the three notations.

Chapter 3

Implementation and Benchmark Setup

3.1 Inside the Native Backend

3.1.1 Parser and Runtime Representation

The parser in `libclingo/src/heuristic_parser.cc` turns each `__heuristic` symbol into a `HeuristicRuleTemplate`: target atom, positive and negative body literals, variable bindings, modifier, semantics, and an arithmetic AST for the weight.

During `init`, the propagator inspects the ground symbolic atoms and builds an index mapping each predicate and its argument tuple to the corresponding solver literal. It then materializes candidates: for each ground target atom and matching combination of positive body atoms, it creates a `RuntimeHeuristicCandidate` storing the target literal, target tuple, body literals, bound variable values, and instantiated aggregate keys.

In BSP with $n = 100$, this yields only 100 candidates (one per `b(X)`), whereas the standard ground encoding produces 1,010,200 directives because it must enumerate all values of S. Thus, candidate materialization scales with the number of ground targets rather than the aggregate domain size, since aggregates are evaluated dynamically at runtime instead of being grounded exhaustively.

Finally, the propagator registers watches exclusively for literals that affect heuristic state: both polarities of target and negative body literals, positive body literals, and aggregate source literals.

3.1.2 Incremental Aggregates

Aggregate states implement three methods: `add`, `remove`, and `result`.

Sums and counts (`SumState`, `CountState`) store a single `int`, because adding and removing values can be done with simple addition and subtraction. In contrast, min and max (`MinState`, `MaxState`) cannot be undone with arithmetic; when the current minimum or maximum is removed during backtracking, the state must know what the next best value was. They therefore use an `std::map<int, int>` that maps each active value to how many times it occurs.

Each source atom updates the `AggregateState` matched by its `RuntimeAggregateKey`: values are added in `propagate` and removed in `undo`.

Under *clingo* semantics, an aggregate is evaluated only after all its source literals are assigned, gating dependent candidates until then. Under *Alpha* semantics, candidates directly read whatever partial aggregate value is currently available on the trail.

3.1.3 Evaluation and Decision

A candidate is *active* when its target literal remains unassigned, all its positive body literals evaluate to true, and every negative body literal is satisfied under the template's semantics. The two supported semantics differ only in how they treat negative literals:

```
static bool negative_literal_is_satisfied(HeuristicSemantics s,
↪  Clingo::TruthValue v) {
    return s == HeuristicSemantics::Clingo ? v == Clingo::TruthValue::False
                                           : v != Clingo::TruthValue::True;
}
```

Once a candidate satisfies its body conditions, the propagator computes its numerical weight by evaluating its weight AST over the current aggregate values and term arguments. The resulting effect is then applied to the target literal's `TargetHeuristicState`.

The native propagator adopts four of clingo's domain heuristic modifiers:

- `level`: sets the branching priority to w (higher values are chosen first), leaving the truth polarity unchanged;
- `sign`: sets the preferred truth polarity based on the arithmetic sign of w (positive for *true*, negative for *false*), using $|w|$ to resolve precedence among conflicting sign directives;
- `true`: sets the branching priority to w and forces the truth polarity to *true*;
- `false`: sets the branching priority to w and forces the truth polarity to *false*.

clingo's remaining modifiers, `init` and `factor`, are not supported because custom propagators cannot manipulate clasp's internal VSIDS score tables.

If multiple heuristic rules target the same atom, their effects are combined into a `TargetHeuristicState` that tracks the `level` and `sign` across all active `RuntimeHeuristicCandidate` instances.

Unassigned targets with an active `level` are stored in an ordered set:

```
std::set<DecisionRankKey, DecisionRankGreater> active_decision_ranks_;
```

Entries are sorted by decreasing weight. Because weights are stored as standard 32-bit integers, this avoids the 16-bit saturation limit of clasp's internal domain table (Section 2.1.2).

When clasp calls `decide`, the propagator inspects `active_decision_ranks_` from the beginning, discarding any targets that are already assigned on the trail or whose weights have changed. It then branches on the first valid target using its stored `sign`. If the set is empty, the propagator checks whether clasp's fallback atom has a stored `sign`; otherwise, it returns 0 to delegate the decision to clasp. During backtracking, `undo` restores aggregates and candidate states using `noexcept` functions to prevent state corruption during solver unwinding.

3.2 The Prolog Backend

While the native backend prioritizes evaluation speed through compiled C++ structures, the Prolog backend evaluates heuristics as arbitrary logic programs via an embedded engine, trading runtime overhead for unrestricted expressiveness. This section describes its runtime architecture and trail synchronization mechanisms (with a comprehensive performance comparison presented in Section 4.14).

3.2.1 Rule Strings and Normalisation

In the Prolog backend, a heuristic is defined as a rule string whose head is a four-argument term: `heuristic(Target, Weight, Priority, Modifier)`. The BSP heuristic from Section 2.3 is specified as follows:

```
heuristic("heuristic(b(X), W, 0, true) :-
    aggregate_all(sum(Y), holds(c(Y)), S), holds(heuristic_base(Base)),
    holds(x(X)), target_available(b(X)), alpha_not(c(X)),
    W is Base*S+X.").
```

The rule body uses standard Prolog syntax and maps directly to the native constructs: `holds/1` for positive conditions, `alpha_not/1` for negative conditions, `aggregate_all/3` for dynamic sums, and `is/2` for arithmetic evaluation of weights. Furthermore, `target_available/1` ensures the target atom is still unassigned. While the native backend checks this implicitly during candidate selection, the Prolog rule must state it explicitly to prevent proposing atoms that are already assigned on the trail.

During initialization, the propagator collects all `heuristic/1` facts, normalizes the rule strings, and classifies their semantics: rules containing `clingo_not(...)` use clingo semantics, while all others use Alpha semantics. The propagator extracts the predicate signatures referenced in the rules (`holds`, `alpha_not`, `clingo_not`, `target_available`, and rule targets) and asserts into Prolog only solver atoms matching those signatures, avoiding unnecessary memory and synchronization overhead from irrelevant predicates.

3.2.2 The Runtime Program

During initialization, the backend starts an embedded SWI-Prolog engine [13] and loads a set of base Prolog rules:

```
holds(A) :- true_atom(A).
holds(A) :- static_atom(A), \+ true_atom(A).
clingo_not(A) :- false_atom(A).
alpha_not(A)  :- \+ holds(A).
target_available(A) :- target_atom(A), \+ true_atom(A), \+ false_atom(A).
dyn_max(Goal, Template, Max) :-
    (aggregate_all(max(Template), Goal, R) -> Max = R ; Max = 0).
```

These rules translate the solver's partial assignment into terms that the heuristic rules can evaluate. As clasp makes decisions, the propagator mirrors the trail in the Prolog database by adding `true_atom(A)` when an atom is assigned to true and `false_atom(A)` when it is assigned to false, removing them upon backtracking. Undecided atoms simply do not exist in the database. Because of this, `alpha_not(A)` succeeds whenever an atom is either false or undecided, implementing Alpha's open-world negation. In contrast, `clingo_not(A)` requires an explicit `false_atom(A)` on the trail. In the same way, `aggregate_all` dynamically computes sums and counts over the atoms currently true on the trail. Finally, `target_available(A)` verifies that a target is neither true nor false before branching on it, and `dyn_max/3` defaults to 0 when evaluating an empty set (Section 2.1.2).

3.2.3 Synchronisation and Decision

Trail synchronisation is incremental: `propagate` and `undo` touch only the atoms mapped by the literals that changed, each with a single combined retract-and-assert goal, because in this backend term parsing and calling — not the logical work — dominate the cost of a synchronisation. At each `decide` the backend enumerates the solutions of `heuristic(T, W, P, M)`, discards candidates whose target is assigned or unmapped, and selects the best by decreasing priority, then decreasing weight, then a deterministic tie-break, with the modifier giving the sign of the returned literal. Note that this is a full re-enumeration per decision, where the native backend maintains a ranked set incrementally; the per-decision instrumentation of Section 4.7 is designed to expose precisely that difference.

The backend is enabled by setting the environment variable `LAZY_HEURISTIC_BACKEND` to `prolog`, and with `LAZY_PROLOG_STATS` set to 1 it prints a summary line with decision counts and cumulative timings of every phase (state synchronisation, Prolog query, candidate scan, selection), which the benchmark parser ingests.

Without the environment variable the same build falls back to an auxiliary evaluator that re-grounds a small clingo program per decision. That fallback accepts a different, ASP-style body syntax, so a Prolog-style rule handed to it simply fails — SWI's `unknown=fail` convention — and the run degenerates to no heuristic at all while still terminating successfully and still producing correct answer sets. The failure is invisible in exit codes and in output, and it is therefore exactly the kind of misconfiguration that quietly corrupts a benchmark campaign; the guards that detect it are described in Section 3.4.

3.3 Benchmark Problems and Encodings

3.3.1 Balanced Set Partitioning (BSP)

BSP partitions $\{1, \ldots, n\}$ into two sets `b/1` and `c/1` whose sums differ by at most one. The guessing part is a symmetric even loop, the check compares the two sums, and the heuristic prefers to place the next element into the set whose current sum is largest, keeping the partition balanced greedily. It is the smallest problem in the suite and the one whose heuristic is purest: a single aggregate, no ordering between atom families, nothing else moving.

That purity has a consequence worth stating, because it is easy to misread the weight. The flattening rule of Section 2.1.2 is *vacuous* on BSP: all its directives belong to one level, since `b/1` and `c/1` are ranked by one and the same criterion. The weight $((n+1) \cdot S) + X$ is therefore not a flattened weight–priority pair but the lexicographic composition of the two components of a single criterion — the dynamic sum S dominating, the element index X breaking ties inside it — with $n+1$ playing the role of the multiplier *within* the criterion rather than between levels. Accordingly all four variants carry priority 0, `la` included, and the `la`/`lc` contrast on BSP isolates the semantics axis alone, with no interference from the priority reading.

Besides the four matrix variants and the baseline, BSP carries three methodological extras: `ga_weak`, which drops the negative literals without the static unrolling and therefore changes only the moment at which the heuristic activates; `la_aux`, which routes the lazy heuristic through an auxiliary predicate `h_b(X,S)` that reintroduces grounded aggregate values, and hence a grounding cost, before the propagator is even reached; and `la_co`, which combines the lazy heuristic with a linear reformulation of the balance constraint, and is treated as a modelling experiment rather than a heuristic comparison.

3.3.2 Partner Units Problem (PUP)

PUP [14] connects zones and sensors to communication units under adjacency and capacity constraints; the instances of the `double` family provide `comUnit/1` and `zone2sensor/2`. The encoding follows the Alpha evaluation encoding with an inductive, locality-based choice structure. The heuristic ranks sensor placements by a four-component weight (dynamic degree, forbidden placements, unit occupancy, direct connections) and zone placements by occupancy and free capacity, with zones globally preceding sensors — so PUP is the benchmark on which the flattening of Section 2.1.2 does real work.

Porting the encoding from Alpha to clingo required four adaptations, each forced by the difference between lazy and eager grounding, and each applied uniformly to *all* variants so that no comparison is affected:

- the inductive `assignable` rules are bounded by `comUnit(U±1)`, because gringo would otherwise materialise an unbounded chain that Alpha explores lazily;
- the existential capacity constraints are rewritten as monotone counting aggregates with linear grounding;

- a redundant distance-two blocking rule with cubic grounding is removed, since it prunes nothing the remaining constraints do not already prune;
- counters are capped at the unit capacity (`sensorNumbers(0..2)`), without which the auxiliary predicate `num_forbidden_places_of_sensors` explodes in every variant and masks the effect under study.

The zones-before-sensors ordering is flattened into the weight in all four variants and in both backends (offset +100,000, against a maximum sensor weight of 26,220), with the priority field left at 0 throughout. The related `doubleV` family is not used here; the reasons are taken up with the other limitations in Chapter 5.

3.3.3 House Reconfiguration Problem (HRP)

HRP [15] extends the House Configuration Problem with legacy configurations: things must be stored in cabinets and cabinets placed in rooms owned by the things' owners, under capacity constraints refined by the thing-length/cabinet-size extension, while a legacy configuration can be partially reused or dismantled. The heuristic is a strict five-level policy: reuse legacy components first (with internal weights), then fill cabinets with long things, then with short things, then place cabinets into rooms, and finally a closing level that biases all remaining choice atoms to false, pruning the tail of the search.

HRP is the negation-heavy benchmark, and that is why it is in the suite: its heuristics contain no aggregates at all, so the `la`/`lc` comparison isolates the effect of the negation semantics alone, whereas in BSP and PUP the two mechanisms act together and cannot be told apart. In the native lazy encodings the guards over the partial state are precompiled into same-tuple auxiliary predicates — for example `cabFull(C,T) :- cabinetDomain(C), thing(T), fullCabinet(C)` — so that negative conditions align with the target tuple as the DSL requires. The five levels are flattened with the instance-derived multiplier `levelMul(LM) :- LM = MC + MR + 10` in all four variants and in both backends. HRP is also the only family whose Prolog encodings use two distinct values in the priority slot, separating the `true` directives from the closing `false` ones on the same target; the flattened weights already induce that order on their own, so the slot is redundant, and the native encodings, whose language has no priority field, obtain the same behaviour from the weights and the modifier alone.

3.3.4 Alpha as an External Reference

The four variants of the matrix all run inside clingo, and by construction they can only answer questions about clingo. One question they cannot answer is the one the thesis exists to raise: what does it cost to obtain the Alpha semantics *inside* clingo, compared with taking it from the system that implements it natively? The benchmark suite therefore defines a third `<system>`, `alpha-qh`, which is not part of the matrix and is never mixed with it.

The system is Alpha in the *Qh* variant, invoked with `-uqh`, in which declarative heuristics are evaluated as Prolog queries against the partial assignment and aggregates inside them are dynamic [9]. This detail is not cosmetic: the official

0.7.0 release does not support `#heuristic` at all, and the upstream branch that accepts the same syntax evaluates the aggregate *statically*, which on BSP makes the heuristic balance nothing and the backtracks explode. Only the Qh build carries the semantics this thesis reproduces. The encodings are copied verbatim from the authors' own materials, with no adaptation, and the PUP instances of the suite are byte-identical to those of the supplementary material, so the comparison is against the reference implementation running its reference encoding.

Two settings are used. `alpha` is that encoding with its heuristic; `alpha_noheur` is the same encoding solved with Alpha's default heuristic (`-ids`), and stands to `alpha` exactly as `gc_noheur` stands to `gc`. Having both matters, because it is what separates the contribution of lazy grounding from the contribution of the heuristic; Section 4.13 reads the four combinations against each other. The comparison covers BSP and PUP, the two families whose reference heuristics carry dynamic aggregates.

What can be compared across the two systems is narrower than within the matrix, and the boundary must be stated before any number is read. Alpha and clasp count different things: Alpha's *guesses* and *backtracks* are choice points of a lazy-grounding CDNL search over a program that is being instantiated as it goes, and its *grounder calls* have no counterpart at all in a ground-and-solve pipeline, so none of these is commensurable with clasp's choices and conflicts. Only the external measures taken by `runlim`, wall-clock time and peak resident set size, are measured the same way for both. Even those carry one asymmetry: Alpha runs on the JVM, and its peak RSS therefore includes the heap *reserved* through `-Xmx` (fixed by the wrapper below the memory limit), not only the heap in use. Alpha's memory figures thus measure the cost of running the system, not the state its algorithm holds, and they are read as an upper bound.

3.4 Correctness Validation

Section 2.2 argued that a heuristic cannot change the answer sets. The argument is sound but it is still an argument about code, and the claim it supports is falsifiable, so the pipeline checks it before measuring anything.

The harness `tools/check_equivalence.py`, run by the sanity entry point ahead of every campaign, performs an *exhaustive equivalence* check on a small BSP instance: it enumerates all models with `clingo -n 0` for every variant and both backends, projects the models onto the solution predicates `b/1` and `c/1`, and requires the projected collections to be identical across variants and across backends. The projection is not cosmetic: the encodings expose different `#show` sets and carry different auxiliary predicates, so comparing raw witnesses would report differences that are artefacts of the instrumentation. For PUP, where exhaustive enumeration is intractable even on small instances, the harness falls back to agreement on SAT/UNSAT together with the choice counts.

The second thing the harness guards against is the silent-fallback failure mode of the Prolog backend described in Section 3.2.1, which is more dangerous than an outright bug precisely because it produces correct results. Every lazy Prolog run must report `decide_calls > 0` in the propagator summary: a run that terminated successfully but never called the custom `decide` has degenerated to the no-heuristic

baseline, and is treated as an error by the harness and flagged with an explicit warning column by the benchmark runner. As a cross-check, the harness verifies that native `la` and Prolog `la` perform the same number of choices on the control instance — if the SWI engine were inactive, the two runs would diverge immediately. The benchmark wrapper exports `LAZY_HEURISTIC_BACKEND=prolog` itself, so the guard protects against configuration drift rather than repairing it.

The C++ unit tests complete the picture on the native side, covering incremental aggregates, corner cases such as empty domains and unsatisfiable instances, explicit body and target bindings, filtered aggregates, the per-channel resolution of the modifiers by weight under both semantics, and the syntax validation of the parser, including the rejection of a `__priority` field.

3.5 Benchmark Infrastructure and Metrics

The campaigns are orchestrated with `benchmark-tool`, driving one run per (system, setting, instance) triple under `runlim` [16] for wall-clock and memory limits. The two systems are shell wrappers around the two modified binaries; both fix `--stats=2 --heuristic=Domain -n 1` and a numeric locale, and the Prolog wrapper additionally exports the backend activation and statistics variables. A custom result parser extracts, for every run: outcome and times (total, solving, and their difference as an estimate of the non-solving part), search statistics (choices, conflicts, restarts), grounding-size measures (ground `#heuristic` directives for the ground variants, lazy heuristic facts for the lazy ones, and rule counts from clasp's statistics), the peak memory, and, for both lazy backends, the propagator summary (decision calls, candidates seen per decision, cumulative synchronisation, query, scan and selection times). The two backends emit that summary through the same parser, so the per-decision cost of the compiled and of the interpreted evaluation are directly comparable; the one metric that is *not* comparable across them is the candidate count, since the native backend reports the candidates it retains after incremental filtering while the Prolog backend reports every solution the `heuristic/4` query returns. Wherever a candidate count appears in Chapter 4 the backend it comes from is named.

Three measurement caveats are declared up front, because they define what the numbers can and cannot show. First, the campaigns run on a shared cluster whose nodes are not identical, and SLURM dispatches each job to whichever node is free. Wall-clock times therefore carry a node-to-node spread that Section 4.2.1 quantifies on programs known to be identical: a median of 28% and a maximum of 46% on the grounding component. The structural counters emitted by clasp and by the propagator — choices, conflicts, rules, atoms, variables, decide calls — are unaffected, and the analysis leans on them wherever a time gap is not an order of magnitude wide. Second, the memory metric is the peak resident set size of the whole solver process, end to end: for the lazy Prolog variants it includes the in-process SWI engine and its asserted database. The engine has a fixed initialisation overhead independent of n, so on small instances the lazy variants may show a *larger* RSS than the ground ones; the lazy memory advantage is asymptotic, emerging where the $\Theta(n^3)$ growth of ground heuristics overtakes that constant. Being a property of

the program rather than of the machine, however, peak RSS does not suffer from the first caveat, which is why it carries the central memory claim of Chapter 4.11. Third, the ground-size comparison counts objects of different kinds: for the ground variants it counts directives actually materialised by gringo, a faithful one-line-one-object count, whereas for the lazy variants it counts template facts that expand at run time into one candidate per ground target. The line count therefore *understates* the run-time work of the lazy backends by design; that work is measured separately by the per-decision metrics. All three caveats are taken up again in the discussion of the results.

Chapter 4

Results

4.1 Functional Validation

The system was validated by keeping three levels separate. The first level concerns the ASP program itself: the answer sets, the `SAT`/`UNSAT` result, and the visible solutions must remain consistent across the compared encodings and across the two backends. The second level concerns the heuristic translation: targets, bodies, weights, modifiers, and aggregate bindings must be preserved when a native `#heuristic` directive is rewritten as a lazy fact, including the flattening of levels into the weight, which the suite applies uniformly to every variant and to both backends. The third level concerns the effect on search, measured through choices, conflicts, solving time, memory, and the size of the ground heuristic component.

Heuristics do not change the semantics of the ASP program: they guide search, but they do not add or remove answer sets. The equivalence harness of Section 3.4 verifies this property exhaustively on BSP, by enumerating all models of every variant on both backends and comparing the collections after projection on `b/1` and `c/1`, and by SAT/UNSAT agreement on PUP, where exhaustive enumeration is intractable. The same harness enforces the anti-fallback guards for the Prolog backend (`decide_calls > 0` for every lazy Prolog run, and equal choice counts between native and Prolog `la` on the control instance). On the native side, the C++ test suite isolates the correctness of the heuristic machinery from the benchmarks: it covers incremental aggregate states, empty and unsatisfiable corner cases, explicit body and target bindings, filtered aggregates, the per-channel resolution of the modifiers by weight under both semantics, and the parser's rejection of malformed syntax.

4.2 Experimental Setup and Dataset

The campaign reported in this section was executed on an HPC cluster under SLURM, with one job per (system, setting, instance) triple orchestrated by `benchmark-tool`. Every run was limited by `runlim` to 600 seconds of real time and 30 GB of resident memory, on 4 dedicated cores. Both backends were compiled on the compute nodes against the same toolchain, and the Prolog backend embeds a modern SWI-Prolog (10.0.2) built in user space for the same environment. The dataset covers the three families with their canonical instance ranges: BSP with

$n = 10$ to 200 in steps of 10; PUP with the `double-$N$` instances, $N = 20$ to 200 in steps of 20 communication units; HRP with the reconfiguration instances `house-$N$`, $N = 2$ to 20 persons in steps of 2. All five shared settings run on all families; the three methodological extras (`ga_weak`, `la_aux`, `la_co`) are BSP-specific. The grand total for the matrix is 520 runs (320 for BSP, 100 each for PUP and HRP), to which the external reference system of Section 3.3.4 adds 60 more (`alpha` and `alpha_noheur` on BSP and PUP), for 580 in the campaign; none of them ended in error: every unsolved run is an explicit timeout (T) or memout (M). Under this raised budget memouts are rare and concentrated: 11 runs out of the 580, all on BSP and all inside the matrix, ten of them at exactly $n = 180$ (five variants on each backend, where the base program itself reaches the ceiling) and one for `la_aux` at $n = 170$; in every one of those runs the time limit was hit as well, so the two flags are raised together and the memout adds no separation. The dominant failure mode throughout the campaign is therefore timeout, and the frontiers reported below should be read as time-bound rather than memory-bound except where stated.

The figures of this chapter follow the same separation. Every comparative plot shows the reference encoding and the four variants under study, `gc_noheur`, `gc`, `ga`, `la`, and `lc`; the three exploratory settings are kept out of these plots on purpose, because each of them answers a different, narrower question, and appear only in the dedicated analyses of Sections 4.8, 4.9, and 4.10, in figures restricted to the variants that each comparison actually needs.

Three properties anchor the validity of the numbers, and one qualifies them. First, within each system the ground variants (`g*`) execute identical binaries on identical ground programs, and clasp's search is deterministic at fixed configuration: choice and conflict counts are therefore exact structural measures, not noisy samples, and they reproduce bit-for-bit the counts observed in the preliminary local campaign (for example, `gc` at $n = 100$ performs exactly 32,514 choices on both machines). Second, the anti-fallback guard confirmed `decide_calls > 0` for every solved lazy Prolog run, so no lazy measurement silently degenerated to the baseline. Third, the two backends agree on the search trajectory wherever the heuristic is total and its ranking admits no ties: on BSP the choice and conflict counts of native and Prolog coincide to the unit in all 102 cells that both backends solve, ground and lazy alike. Where ties do occur the agreement is close but not exact — on PUP and HRP the lazy variants differ by a few percent in choices (for instance 217 against 197 for `la` at PUP $N = 160$), because the deterministic tie-break of the two implementations, a literal ordering in C++ and a sort key in Prolog, resolves equal-rank targets differently. All ground cells, on every family, remain bit-identical across backends.

4.2.1 What the Times Can and Cannot Carry

The qualification concerns *time*, and it must be stated before any number below is read. Jobs are dispatched by SLURM to whichever node of the `kr,kr-big` partitions is free, and those nodes are not identical. The effect is directly measurable, because three BSP variants (`gc_noheur`, `la`, `lc`) share the base encoding verbatim and therefore ground the same program to the rule (at $n = 120$, 52,759,256 rules against 52,759,258, the two extra being the lazy facts themselves): their grounding times

should coincide, and instead they spread by a median of 28% and by as much as 46% at $n = 140$ (341.2, 336.1 and 234.1 seconds for the same program) and 38% at $n = 120$. At small sizes the spread is irrelevant in absolute terms; from $n \approx 100$ upwards, where grounding costs hundreds of seconds, it is large enough to reorder same-encoding variants in a time plot and, at the very top of the range, to decide on which side of the 600-second limit a run falls. Consequently: structural measures (choices, conflicts, rules, variables, atoms, ground objects, decide calls, candidates) are exact and are the basis of every claim in this chapter; time measures are reported as observed and are load-bearing only where the gap between variants is far larger than this spread, as it is for `ga` against `la` on PUP (a factor of two) or for `lc` against `la` on HRP (two orders of magnitude), and not where it is comparable to it, as it is between same-encoding variants at the top of the BSP range. Peak RSS is unaffected: it depends on the program, not on the node, and the same three variants agree to within 0.2% at every size from $n = 70$ upwards, where memory is large enough for the comparison to mean anything (the largest disagreement anywhere is 2%, at $n = 40$, on runs of under a hundred megabytes).

Tables 4.1–4.3 summarize the outcome of every cell of the experimental matrix. The columns report how many instances were solved and where the first failure occurs, distinguishing timeouts from memouts.

	native		**Prolog**	
Variant	**solved**	**first failure**	**solved**	**first failure**
`gc_noheur`	15/20	T@160	16/20	T@170
`gc`	13/20	T@140	15/20	T@160
`ga`	6/20	T@70	6/20	T@70
`ga_weak`	16/20	T@170	16/20	T@170
`la`	15/20	T@160	17/20	T@180
`lc`	15/20	T@150 (non-mon.)	15/20	T@160
`la_aux`	6/20	T@70	3/20	T@40
`la_co`	20/20	—	20/20	—

Table 4.1: BSP campaign overview ($n = 10..200$, 600 s / 30 GB per run). At $n = 180$ five of the eight variants fail with both flags set (T and M simultaneously) on both backends, the one size at which the base program's own growth exhausts the memory budget at essentially the same point at which it would time out; `la_aux` reaches that point one size earlier. One native cell is non-monotone (`lc` fails at 150 but solves 160). Above $n \approx 130$ all same-encoding variants are within the node-to-node spread of the grounding time discussed in Section 4.2.1, so the exact position of their frontiers should not be over-read: whether `la` stops at 150 or at 170 is a property of the nodes it was dispatched to, not of the variant. The separations that survive that spread are `ga` and `la_aux` at one end and `la_co` at the other.

	native		Prolog	
Variant	**solved**	**first failure**	**solved**	**first failure**
`gc_noheur`	4/10	T@100	4/10	T@100
`gc`	3/10	T@80	3/10	T@80
`ga`	10/10	—	10/10	—
`la`	10/10	—	10/10	—
`lc`	4/10	T@80 (non-mon.)	2/10	T@60

Table 4.2: PUP campaign overview (`double-`N, $N = 20..200$, 600 s / 30 GB per run). Under this budget neither `ga` nor `la` fails anywhere in the tested range; the two variants whose heuristic carries no dynamic aggregate (`gc_noheur`, `gc`) stall by timeout well inside it. One native cell is not monotone in N: `lc` fails on `double-80` but solves `double-100`. This is a property of the family — one instance per size, with neighbouring sizes of very different hardness — and Section 4.11 reads it through the choice counts rather than the times.

	native		Prolog	
Variant	**solved**	**first failure**	**solved**	**first failure**
`gc_noheur`	10/10	—	10/10	—
`gc`	10/10	—	10/10	—
`ga`	10/10	—	10/10	—
`la`	10/10	—	10/10	—
`lc`	8/10	T@18	7/10	T@16

Table 4.3: HRP campaign overview (`house-`N, $N = 2..20$ persons, 600 s / 30 GB per run).

Three headline facts are already visible at this granularity and are unpacked in the following subsections. The first is that the three families put the frontier in three different places, and only two of them are informative. On PUP the frontier separates the variants by more than a factor of two in reach and is set by the heuristic's own grounding: it is the family where the comparison is decided. On HRP no variant except `lc` is ever in difficulty, so the frontier says nothing and the guidance quality says everything. On BSP the frontier of every same-encoding variant is set by the grounding of the base program, which the heuristic representation does not touch: `gc_noheur`, `la` and `lc` all stop between $n = 150$ and $n = 170$ depending on the backend and on the node the job landed on, within the node-to-node spread of Section 4.2.1, and the ordering among them is not a stable property of the campaign. What *is* stable on BSP, because it is structural, is that `la` reaches that common wall having performed exactly n choices and zero conflicts at every size, that is, with the search cost removed entirely rather than reduced.

The second fact is that two variants separate themselves from that band by a margin far larger than any measurement spread, and they do so in opposite directions. The ground Alpha simulation `ga` collapses on BSP precisely because of the static unrolling that makes it faithful, failing at $n = 70$, nine sizes before the

heuristic-free baseline; and `la_co` solves the entire range in milliseconds. Third, the clingo-like lazy variant `lc` exhibits two distinct failure modes, inertness on BSP and active misguidance on HRP, that the per-decision instrumentation makes directly measurable; it is also the variant whose frontier differs most between the two backends on the families where the propagator is consulted tens of thousands of times (PUP and HRP), precisely because it is the one whose cost is dominated by the per-decision machinery.

4.3 Reduction of the Ground Heuristic Component

The central claim of the thesis concerns *where* memory is spent, and the direct evidence is the number of ground heuristic objects as a function of n. The mechanism is readable in the encoding itself. The standard directive

```
#heuristic b(X) : x(X), not c(X), S = #sum{Y : c(Y)},
    W = ((n+1)*S)+X. [W, true]
```

carries the aggregate value S *as a variable inside the weight*. Since the atoms `c(Y)` are choice atoms, the grounder cannot know the sum at grounding time and must materialize one directive for every reachable pair (X, S). With $X \in \{1, \ldots, n\}$ and S ranging over the subset sums of $\{1, \ldots, n\}$, which is every integer from 0 to $n(n+1)/2$ because 1 belongs to the domain, the count is

$$2 \cdot n \cdot \left(\tfrac{n(n+1)}{2} + 1 \right) \;=\; \Theta(n^3),$$

where the factor 2 accounts for the two symmetric rules for `b` and `c`. The formula is verifiable exactly on the measured data: for $n = 10$ it yields $2 \cdot 10 \cdot 56 = 1{,}120$, which coincides with the measured `ground_heuristics` value for `gc`. The explicit unrolling of `ga` materializes the same (X, S) product by construction. The lazy variants remove this enumeration at the root: the aggregate is evaluated at solving time on the current partial assignment, and the materialized heuristics remain constant, two facts, independent of n.

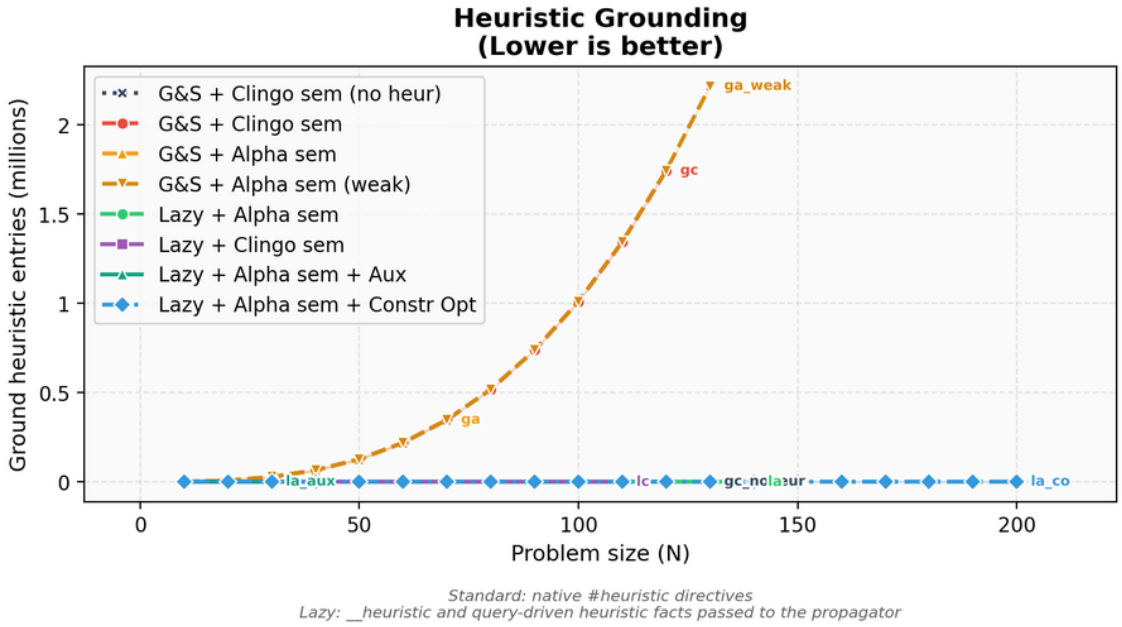

Figure 4.1: Comparison between native ground heuristic objects and lazy heuristic facts.

n	Native heuristic objects	Lazy facts	Ratio
10	1,120	2	560×
50	127,600	2	63,800×
100	1,010,200	2	505,100×
110	1,343,320	2	671,660×

Table 4.4: Growth of the native heuristic component compared with the lazy representation. The counts are properties of encoding and instance, independent of the machine.

This comparison must be read for what it is. For the ground variants the count is faithful: one line is one materialized object. For the lazy variants the two counted facts are *templates* that expand at runtime into one candidate per ground target (n candidates per rule for BSP); those candidates never pass through the grounder and therefore appear in no line count. The single-figure comparison "2 facts versus $\Theta(n^3)$ directives" is thus the proof of a saving in *grounding space*, not of a saving in total work: the work moved to runtime is real and is measured separately in Section 4.7.

4.4 BSP: Runtime Behaviour

Figure 4.2 reports the total time as measured by clingo, and Figure 4.3 its solving component. The base program dominates the picture for every same-encoding variant: at $n = 120$ the domain encoding grounds into 52.8 million rules, a component that `gc_noheur`, `la` and `lc` pay identically, since they share the encoding verbatim except for the heuristic representation (measured at 164.1, 161.6 and 118.7 seconds

respectively, the difference being the node spread of Section 4.2.1 on a quantity that is provably the same). The total-time plot therefore has a wide band at the top of the range in which the same-encoding curves cross each other without meaning; what separates the variants, and what the next figure isolates, is the solving component and, for the ground heuristics, the additional grounding of the directives themselves.

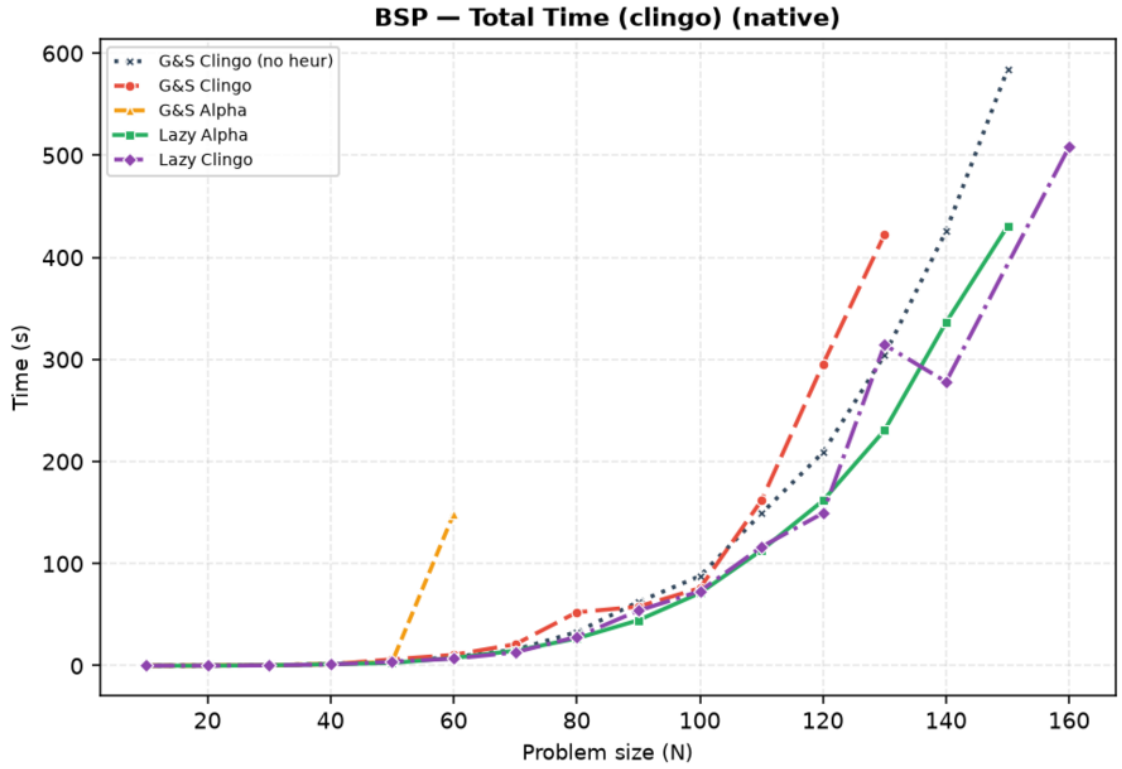

Figure 4.2: BSP, native backend: total clingo time. One run per point; a curve ends where its variant stops solving within the limits. Above $n \approx 120$ the three same-encoding curves (`gc_noheur`, `la`, `lc`) cross each other repeatedly: they ground identical programs, and the crossings are the node-to-node spread of Section 4.2.1, not a difference between variants. Figure 4.3 isolates the component that does separate them.

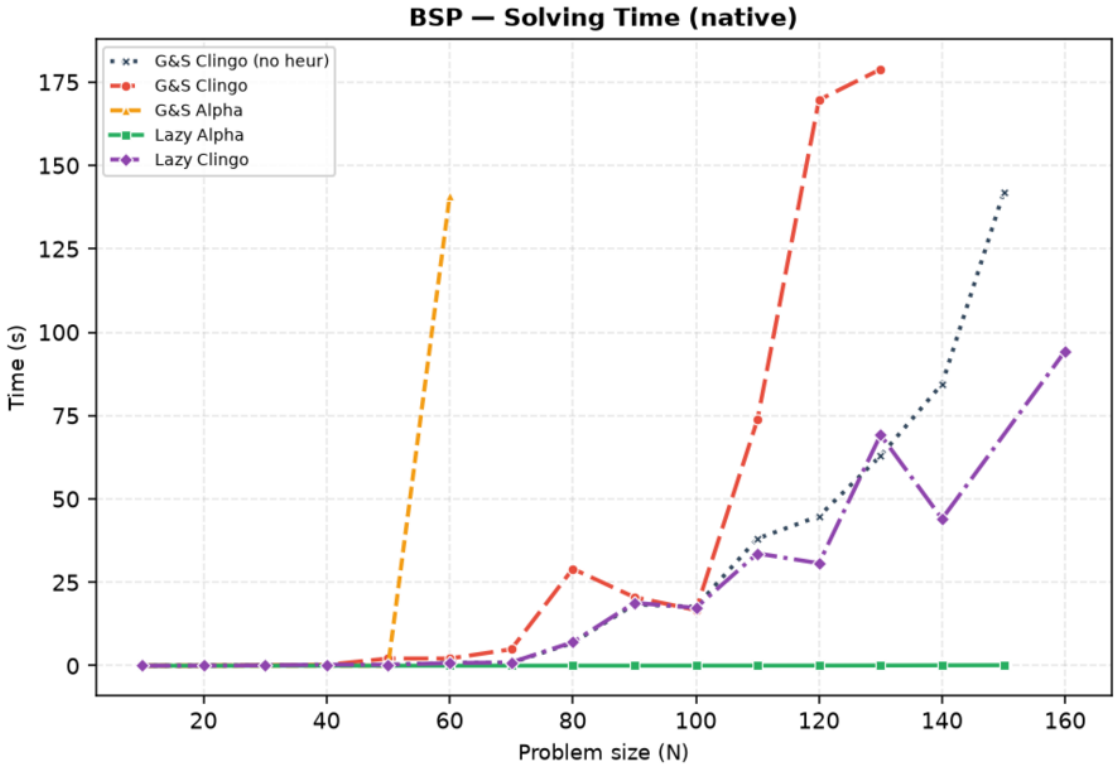

Figure 4.3: BSP, native backend: solving time only. `la` is indistinguishable from zero at every size; `ga` leaves the chart after $n = 60$.

Variant	Total (s)	Grounding est. (s)	Solving (s)	Choices	Conflicts
`gc_noheur`	208.9	164.1	44.8	78,565	49,993
`gc`	295.3	125.7	169.6	231,260	160,115
`ga`	T	(169.2)	(>430.8)	736,888	539,340
`ga_weak`	164.1	121.4	42.7	121,159	80,847
`la`	161.7	161.6	0.04	120	0
`lc`	149.4	118.7	30.7	78,565	49,993
`la_co`	0.027	0.027	0.00	117	0

Table 4.5: BSP at $n = 120$, native backend, under the 600 s / 30 GB limits. T marks a timeout; the parenthesized solving value for `ga` is a lower bound (total minus grounding at the moment it was killed), and its choices/conflicts are the counts reached before the kill. `gc_noheur`, `la` and `lc` ground programs of the same size (about 52.76 million rules), and `gc`, `ga` and `ga_weak` programs of another common size (about 54.5 million): differences in the grounding column within each group are node spread, not variant cost, and only the solving column separates same-encoding variants reliably — and even there, a gap of a few seconds between `gc_noheur`, `ga_weak` and `lc` carries no signal. Choices and conflicts are exact. `la_aux` is discussed in Section 4.9.

Three observations follow from Table 4.5. First, `la` turns search into a formality, 120 choices and zero conflicts, so its total time *is* the grounding time of the base

program, to four decimal places (161.667 against 161.627). This is the strongest form the claim can take on this family, and it is stronger than a comparison of totals: the solving column, which is the only one the heuristic controls, drops from 44.8 seconds on the baseline to 0.04, a factor of a thousand, while every other cost in the run is left exactly where the encoding put it. Under the raised time budget `gc` now finishes at $n = 120$ as well, but pays 169.6 seconds of solving where `la` pays 0.04, and it is `gc`, not `la`, that first crosses into timeout territory as n grows further (Table 4.1: `gc` fails from $n = 140$, `la` from $n = 160$, at which point the base program alone consumes the entire budget).

Second, `ga` is the surprise of the full campaign, and a negative one. Its static unrolling was designed as the faithful ground simulation of the Alpha reading, and up to $n = 50$ it is exactly that: n choices and zero conflicts at every size, indistinguishable from `la`. It then collapses. At $n = 60$ it spends 140.9 seconds of solving against 0.84 of the baseline, with 1,668,059 choices against 11,810, and from $n = 70$ it never finishes within the limit even at 600 seconds (at $n = 120$ it was killed at 736,888 choices and counting). This is a gap of more than two orders of magnitude on a structural measure, so it is entirely unaffected by the caveat of Section 4.2.1.

The cause is not the grounding. Table 4.4 shows that `ga` materialises the same $\Theta(n^3)$ directives as `gc`, and at $n = 70$, the first size it fails, it grounds in 15.3 seconds and 881 MB against the 14.7 seconds and 845 MB of `la`: half a second and thirty-six megabytes, on a budget of 600 seconds and 30 GB. The cause is the weight bound of Section 2.1.2. The unrolled directives carry the current value of the aggregate *in the weight*, and from $n = 51$ that value no longer fits the field clasp reserves for it, so the upper part of the sum range saturates at a single level and the reconstruction of the dynamic value stops working. Section 4.8 isolates the mechanism and shows that the same encoding, rescaled to stay inside the bound, solves $n = 70$ in 665 choices. The datum therefore supports a sharper conclusion than a cost argument: inside a ground-and-solve pipeline a dynamic aggregate is not merely expensive to simulate, it ceases to be representable once the instance grows, which is why `ga` is the only variant of the four-variant matrix, ground or lazy, that fails before the heuristic-free baseline does, and by nine sizes (the exploratory `la_aux` fails there too, for the unrelated reason analysed in Section 4.9).

Third, Table 4.5 contains two rows that the figures of this section deliberately omit. `ga_weak`, the cheap approximation of the Alpha reading, stays close to the baseline and is compared against the faithful `ga` in Section 4.8; `la_co`, which changes the problem model itself, is the subject of Section 4.10.

4.5 BSP: the Two Semantics Under the Microscope

Search statistics separate the two semantics sharply, and the full campaign adds an instrumentation-level proof of *why*. The `la` variant makes exactly n choices with zero conflicts at every size: the Alpha reading makes the greedy balance heuristic total, one direct decision per element, which is the known optimal strategy for this problem. The `lc` variant is not merely "close to the baseline": it is *search-identical* to it. At every size, its choice and conflict counts coincide with `gc_noheur` to the last unit (78,565 and 49,993 at $n = 120$).

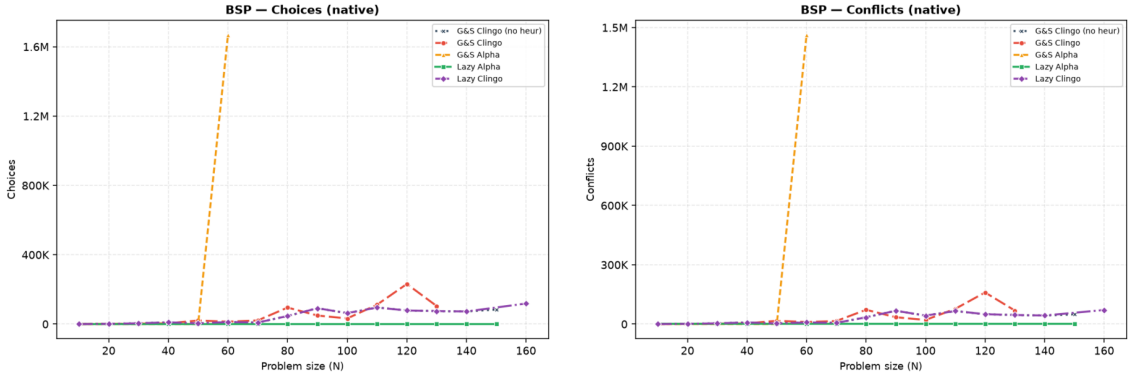

Figure 4.4: BSP, native backend: choices and conflicts. `la` lies on the horizontal axis; `lc` coincides with the baseline exactly, at every size both solve, so the two curves are superimposed; `ga` leaves the chart after $n = 60$. Unlike the time plots, these are exact structural counts.

The instrumentation explains the coincidence, and both backends now report it independently. On BSP the `lc` rule guards the aggregate source with the clingo reading: the sum over `c/1` becomes usable only when every `c` atom is determined, and `clingo_not(c(X))` additionally requires established falsity. During search these conditions are essentially never satisfied at decision time: at $n = 120$ both backends record exactly 78,565 decide calls — one per solver choice, the propagator is consulted every single time — with an average of *zero* applicable candidates per call (`avg_candidates_per_decide` = 0.0, and `max_candidates_seen` = 0, so not a single call in the whole run produced a candidate). Every decide call returns nothing, the solver falls back to its default heuristic, and the trajectory reproduces the baseline exactly. The clingo-like lazy variant on BSP is therefore *inert by semantics*: it costs the full decide-loop machinery and buys nothing, which is precisely what the design predicts for a heuristic whose guards wait for information that only exists at the bottom of the search tree.

4.6 BSP: Rules, Variables, and Memory

The compression of the heuristic component is visible in the size of the propositional instance. At $n = 100$, `gc` is fifty times larger in solver variables than every other variant, because its heuristic directives materialize the negative body and the aggregate bound for each (X, S) pair. Notably, `ga` does *not* inflate the variable count (20,402, in line with the baseline): its unrolled directives reuse the same aggregate structures, and its cost shows up as grounding time and unguarded activations instead. The two ground variants thus pay the same combinatorial product in two different currencies.

Variant	Variables at $n = 100$
`gc`	1,030,606
`ga`	20,402
`ga_weak`	20,406
`lc`	20,300
`la`	20,300
`gc_noheur`	20,300

Table 4.6: Number of solver variables at $n = 100$, native backend.

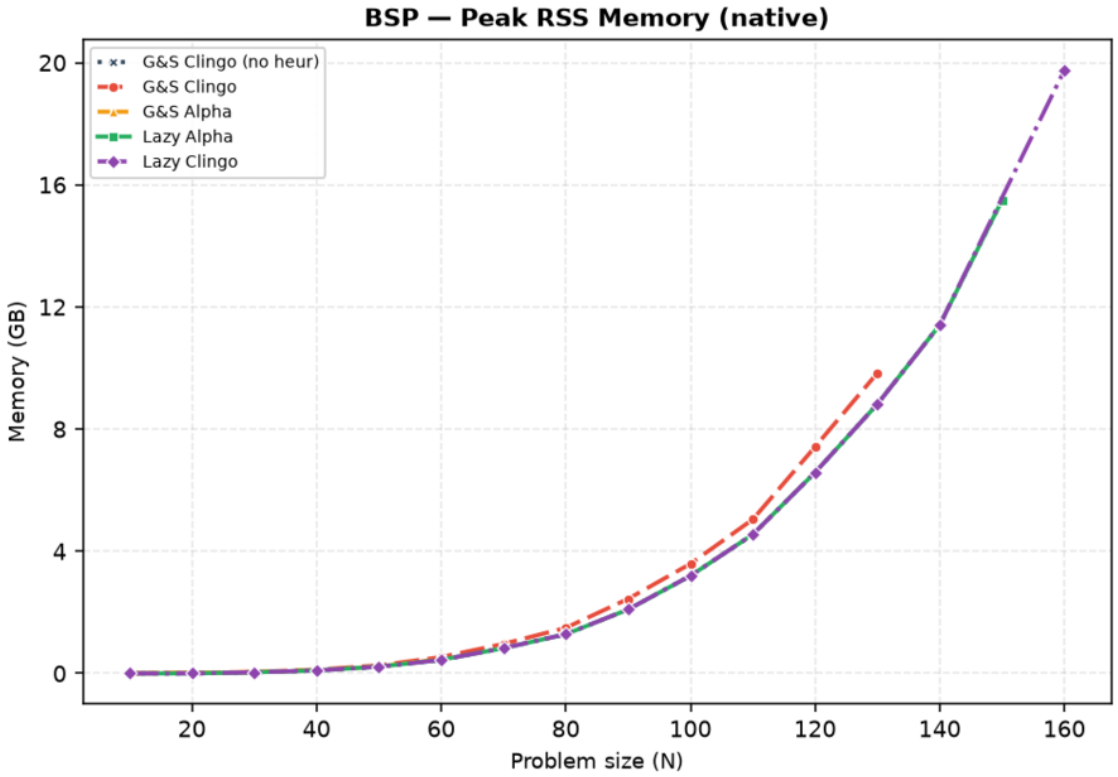

Figure 4.5: BSP, native backend: peak resident set size.

The memory metric is the peak resident set size of the entire solver process, end to end: it covers grounding, CDCL search, and the custom propagator in a single value, and for the lazy Prolog variants it includes the in-process SWI-Prolog engine and its asserted database. Unlike the time measures, it is stable across nodes: the three same-encoding variants agree to within 0.2% at every size from $n = 70$ upwards, which is itself a useful control on the metric. At large n memory is dominated by the base program for every same-encoding variant: at $n = 120$, `gc_noheur`, `la`, and `lc` sit at 6.74, 6.74 and 6.74 GB, while `gc` reaches 7.6 GB and `la_aux` 8.7 GB. Under the 30 GB budget none of these is close to the ceiling at this size; the base program only approaches it much further out, at $n = 180$, which is where Table 4.1 records the campaign's only memouts. The heuristic saving is real but bounded by the base program's own footprint; the variant that changes the footprint by orders of

magnitude is `la_co` (Section 4.10), because it changes the base program itself. On the Prolog side the SWI engine adds a fixed overhead of the order of ten megabytes, invisible at this scale and dominant only on the trivial `la_co` runs. The RSS *confirms* the saving on the real consumption; the ground-object count of Table 4.4 *explains its cause*.

4.7 The Price of Laziness: Per-Decision Cost

Intellectual honesty requires stating that the lazy approach does not *eliminate* the cost of the combinatorial explosion: it *moves* it. What ground-and-solve pays once, in memory, at grounding time, the lazy backend pays repeatedly, in time, at every decision. The full campaign turns this from a declared principle into measured numbers, and both backends are now phase-timed, so the cost can be attributed rather than inferred: the propagator reports, per run, its decide calls, the candidates each call produced, the cumulative time spent answering and the cumulative time spent keeping its state synchronized with the trail.

				ms/decide			
Family	**Variant**	**decide calls**	**cand./decide**	**native**	**Prolog**	**sync (s)**	**prop. share**
BSP n=120	`la`	120	121	$1.2{\cdot}10^{-4}$	0.50	0.004	0.1%
BSP n=120	`lc`	78,565	0.0	$0.8{\cdot}10^{-4}$	0.21	3.39	8.7%
PUP N=160	`la`	197	5.3	$4.5{\cdot}10^{-4}$	3.89	8.81	7.3%
PUP N=200	`la`	249	5.2	$5.4{\cdot}10^{-4}$	6.74	16.54	7.3%
HRP N=20	`la`	106	1,248	$1.5{\cdot}10^{-4}$	9.71	0.036	67.8%
HRP N=14	`lc`	73,143	239	$0.9{\cdot}10^{-4}$	3.57	11.54	99.4%

Table 4.7: Propagator instrumentation on representative solved runs. *decide calls* and *cand./decide* are taken from the Prolog backend, which counts every candidate the `heuristic/4` query returns; *ms/decide* is the average time of one consultation, reported by both backends; *sync* is the cumulative cost of mirroring the solver trail into the Prolog database; *prop. share* is the fraction of the Prolog run spent inside the propagator (whole decide callback plus synchronization). The native backend's own synchronization is not shown because on these runs it never exceeds 2.3 seconds, and on the well-guided `la` cells it stays under a second.

Table 4.7 decomposes the overhead along its two multiplicative factors: how often the heuristic is consulted (decide calls, driven by the search trajectory) and how much each consultation costs (driven by the number of candidates the rules generate and by the size of the synchronized state). Before reading the corners, one figure dominates the table and deserves to be stated on its own: the per-consultation cost differs between the two backends by three to four orders of magnitude, from around 10^{-4} milliseconds for the compiled C++ evaluation against 0.2 to 9.7 milliseconds for the SWI-Prolog one. This is the price of generality, measured, and it is the reason the two backends occupy different points of the design space rather than competing on the same one.

The four corners of the space are all realized in the data. `la` on BSP is the benign corner: 120 consultations of half a millisecond each, 63 milliseconds of propagator

time on a run of two minutes, that is 0.05% of it — the clearest demonstration in the campaign that when the heuristic guides perfectly, even an entire embedded logic-programming engine is free. (The two backends' *totals* on that cell differ by far more than 63 milliseconds, 161.7 against 122.2 seconds, but the difference is the node spread of Section 4.2.1 on the grounding they share, not the propagator; this is exactly the situation in which a total-time comparison must not be read.) `lc` on BSP multiplies a *small* per-call cost by 78,565 inert consultations; the query cost stays low (a fifth of a millisecond each), but the count is large enough that the propagator ends up owning 8.7% of the run — 14.8 seconds of queries and 3.4 of synchronization — for a heuristic that never contributes a single decision. `la` on HRP multiplies *few* consultations by a large per-call cost, since the HRP reference rules [8] enumerate more than a thousand candidates per decision; on sub-second instances the 1.03 seconds of query time it accumulates *are* the run (67.8% of it), turning a 0.40-second native run into a 1.54-second Prolog one (×3.9). `lc` on HRP combines both factors, 73,143 consultations at 3.6 milliseconds each: 248 of its 275 seconds go into the queries and 12 more into synchronization, 99.4% of the run, against 1.78 seconds for the native backend on the same instance — the largest cross-backend gap in the campaign (×154). Synchronization, finally, scales with the trail activity and the number of watched atoms and is the one component that grows with instance size rather than with search: on PUP `la` it dominates the query cost by an order of magnitude (8.81 against 0.76 seconds at N=160, 16.54 against 1.67 at N=200), which is what makes the batched-update protocol of the future work a quantified target rather than a guess.

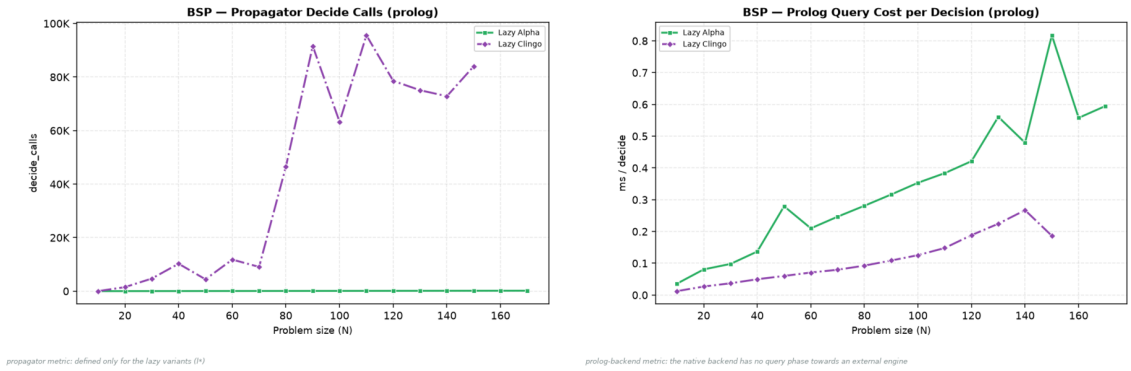

Figure 4.6: BSP, Prolog backend: decide calls and per-decision query cost. The `lc` curve pairs tens of thousands of calls with a low unit cost; `la` the opposite.

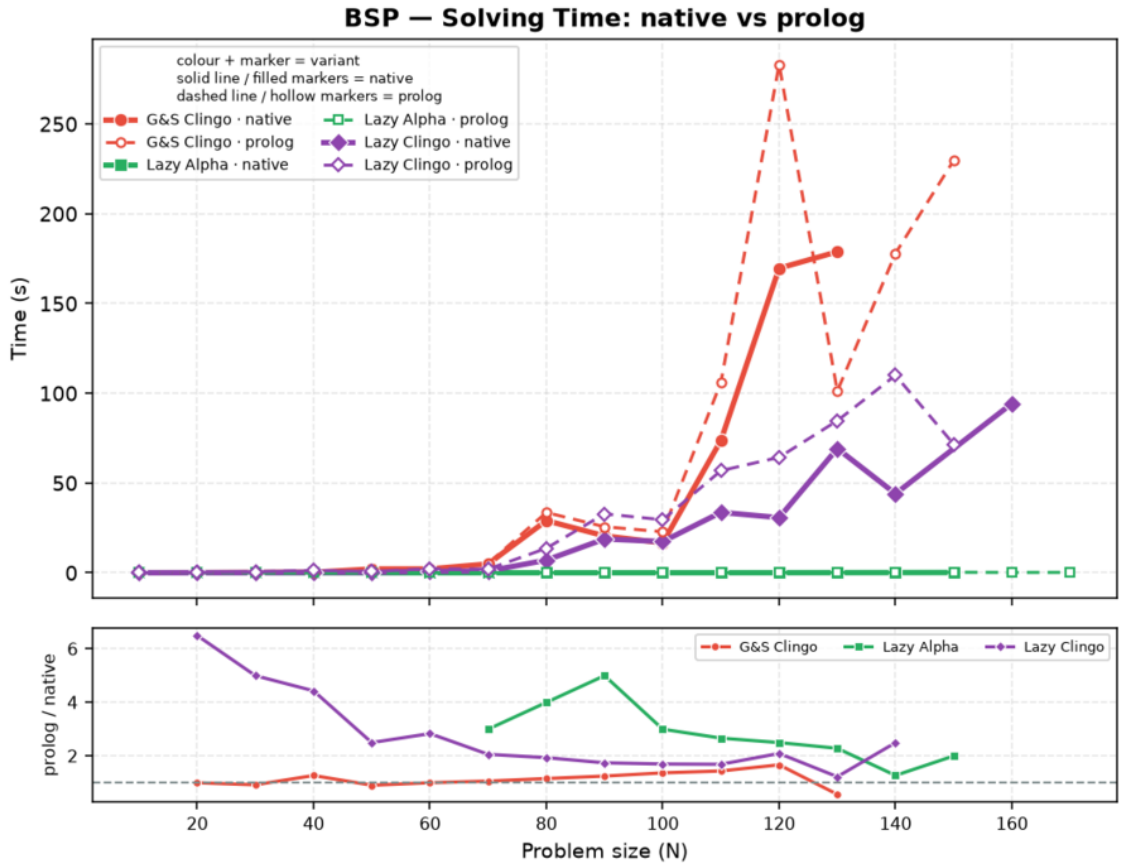

Figure 4.7: BSP: solving time, native (solid) versus Prolog (dashed) backend, lazy variants and ground reference.

The general law suggested by the table, and consistent with all 520 runs, is that the lazy overhead is the product *decide calls* $\times$ (*query cost* + *amortized sync*), where the first factor is governed by how well the heuristic guides (a good heuristic consults itself n times; a bad or inert one, once per baseline choice) and the second by how much of the program the rules can see. The practical reading is that *lazy heuristics are cheapest exactly when they work*: effective guidance keeps the decision count, and therefore the total overhead, small.

4.8 Two Ground Simulations of Alpha: `ga` versus `ga_weak`

The three subsections that follow analyze the exploratory settings. They are not alternative candidates in the main comparison: each one isolates a single design decision, and its figures include only the variants that decision touches.

The first decision is how faithfully the Alpha reading can be forced into a ground-and-solve pipeline. The two transformations of Section 2.1.1 are separable, and `ga_weak` applies only the first: it drops the negative literals, so that established truth no longer deactivates the directives, but keeps the equality binding $S = \#\texttt{sum}\{Y :$

$c(Y)\}$. `ga` applies both, replacing that equality with the monotone bound $S \leq \#\texttt{sum}\{Y : c(Y)\}$ unrolled over every candidate value, so that clasp's max-weight selection reconstructs the current value of the growing sum at solving time.

The distinction is *not* one of grounding size, and this is worth stating because it is the natural guess. All three ground variants materialise the same number of heuristic directives: at $n = 10$, $n = 20$ and $n = 30$ the aspif output of `gc`, `ga` and `ga_weak` contains exactly 1,120, 8,440 and 27,960 heuristic statements respectively, matching the $\Theta(n^3)$ formula in every case. Keeping the equality binding does not spare the grounder anything, because the sum ranges over the same subset sums either way. What the two forms differ in is *when* a directive becomes applicable during search: under the equality binding at most one value of S ever matches, and only once the sum is determined, whereas under the monotone bound every directive whose bound the partial sum has already passed is applicable at the same time.

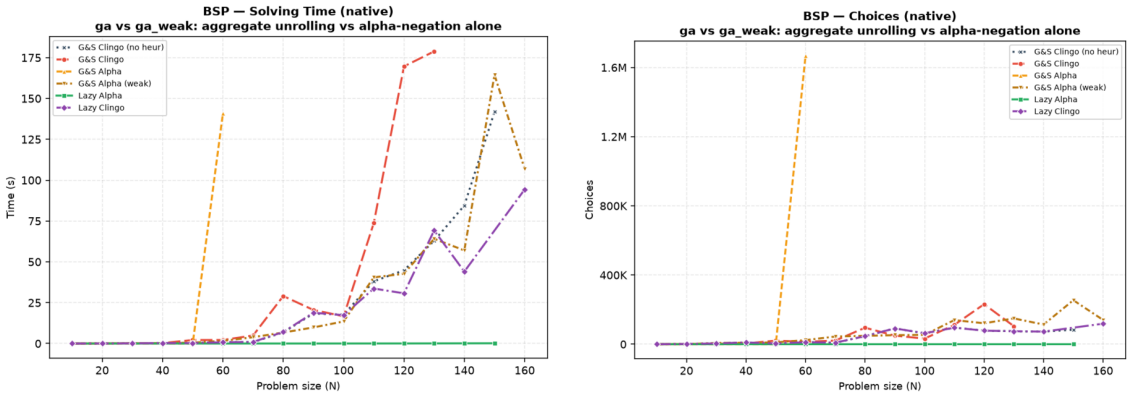

Figure 4.8: BSP, native backend, exploratory comparison: solving time (left) and solver choices (right) with both Alpha simulations alongside the main variants. `ga` is exact up to $n = 50$, degrades from $n = 51$ once its weights leave the representable range, needs 1.7 million choices and 141 seconds of solving at $n = 60$, and times out from $n = 70$; `ga_weak` tracks the baseline over the whole range, and reaches $n = 160$ on both backends.

Figure 4.8 shows what each form costs at solving time. `ga_weak`, which keeps the equality binding, behaves like a heuristic that almost never fires: its solver variables stay at baseline level (20,406 against 20,300 at $n = 100$, since the equality binding needs no auxiliary structure per bound), and its search stays in the baseline's band: at $n = 120$ it explores a noisier trajectory, 121,159 choices against 78,565, for a solving time of 42.7 seconds against 44.8 — a difference well inside the spread of Section 4.2.1 and therefore not a difference at all. The structural counter is the one that speaks here, and it says the extra choices buy nothing. Dropping the negative guards on its own therefore neither guides nor obstructs to any useful degree: it costs the full $\Theta(n^3)$ grounding of `gc` and buys, at best, nothing.

`ga`, which also unrolls the bound, is the variant that actually reproduces the dynamic aggregate, and up to $n = 50$ it does so perfectly: n choices and zero conflicts at every size, exactly like `la`. From $n = 51$ it degrades, and by $n = 60$ it needs 1,668,059 choices and 140.9 seconds of solving against 11,810 choices and

0.84 seconds of the baseline; from $n = 70$ it never finishes within the 600-second limit. The break is not caused by the grounding, which at $n = 70$ costs 15.3 seconds and 881 MB against 14.7 seconds and 845 MB for `la`, a difference of half a second and thirty-six megabytes on a budget of 600 seconds and 30 GB. It is caused by the weight bound of Section 2.1.2. The BSP weight $(n+1) \cdot S + X$ has to represent every value the greedy sum can reach, roughly $n(n+1)/4$ per side, so the ranking survives only while $n(n+1)^2 \leq 4 \cdot 32{,}767$, that is up to $n = 50$. Beyond that the upper part of the aggregate range saturates at a single level and the reconstruction of the dynamic value stops working, first partially and then almost entirely. Rescaling the same encoding so that its weights stay inside the bound confirms the diagnosis: `ga` then solves $n = 70$ in 665 choices instead of timing out.

Read together, the two simulations delimit the ground approach from both sides. The half of the Alpha reading that is cheap to obtain, the activation rule of `ga_weak`, is behaviourally inert. The half that reproduces the guidance, the unrolled aggregate of `ga`, works only while the reconstructed value is representable in clingo's weight field, and neither form spares the grounder anything. The lazy propagator obtains the same guidance without the unrolling and without the bound, which is the design point it occupies: at $n = 120$, 0.04 seconds and 120 choices for the aggregate evaluated lazily, 42.7 seconds and 121,159 choices for the same aggregate bound by equality at grounding time, and a timeout for the same aggregate unrolled.

4.9 Direct and Auxiliary Forms

The `la_aux` variant routes the lazy heuristic through auxiliary predicates that materialize the heuristic body, including the aggregate value, before the propagator is used. This experiment separates two effects that are otherwise easy to confuse: reducing the number of ground heuristic directives, and avoiding the materialization of the heuristic body itself. `la_aux` keeps only two `__heuristic` facts, but its auxiliaries `h_b(X,S)` and `h_c(X,S)` reintroduce the (X, S) product in the base program, and the full campaign shows the consequences from three angles at once. In the grounder, the auxiliary atoms inflate the instance (145,600 solver atoms and 132,756 variables at $n = 50$, against 10,352 and 5,150 for `la`). In the propagator, every (X, S) atom becomes a runtime candidate, so the per-decision scan grows with n^2: the native backend already spends 279.6 seconds of *solving* to finish $n = 60$ (against 0.00 for `la`), and from $n = 70$ it times out at the raised 600-second limit; its peak RSS keeps climbing with n regardless (8.7 GB already at $n = 120$, on its way to a memout at $n = 170$ where the auxiliary relation alone puts five million variables into the propositional instance). On the Prolog side the same candidate flood must additionally cross the synchronization bridge, and there it becomes the whole run: at $n = 20$ it already needs 2.9 seconds of cumulative synchronization for 573 decide calls, against 0.3 milliseconds for the 20 calls of `la`, and at $n = 30$ synchronization alone accounts for 384 of the 387 seconds of the run (99.2%), with the queries themselves costing 24 milliseconds in total. The variant times out from $n = 40$. The signature is worth naming, because it is the opposite of the HRP one: there the propagator was expensive because each query was expensive; here each query is *cheap* (0.02 ms, the cheapest in the campaign) and the cost is entirely in keeping a state that

the auxiliary relation made large. The lesson is clear: if the heuristic body is first materialized into a large auxiliary relation, the intended benefit of laziness is lost on every axis at once, and it is the search cost of the reintroduced product, not memory alone, that dominates the failure. The direct lazy form is the meaningful one.

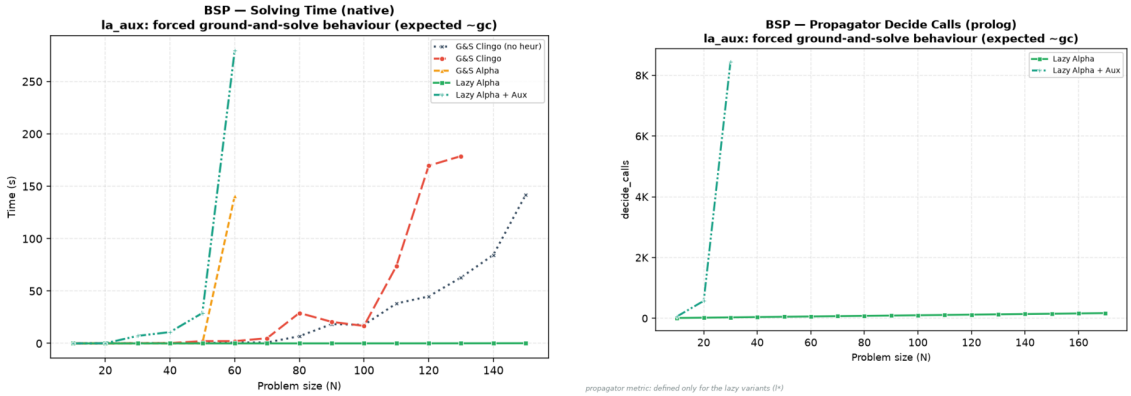

Figure 4.9: BSP, exploratory comparison of the auxiliary form. Left: native solving time; `la_aux` diverges from $n = 40$ and leaves the chart after $n = 60$, while the direct `la` stays on the axis. Right: propagator decide calls on the Prolog backend; the auxiliary candidates multiply the consultations (573 at $n = 20$ and 8,452 at $n = 30$, against the n calls of the direct form), and the curve ends at $n = 30$, the last size the variant solves on that backend.

4.10 Effect of the Constraint Formulation

The `la_co` variant changes not only the heuristic representation but also the BSP constraint itself, replacing the quadratic difference check over `sumB`/`sumC` with a single interval constraint over one sum, whose bounds are compile-time constants. The effect is dramatic and worth stating exactly: at $n = 120$ the propositional instance drops from 52,759,258 rules to 366 and from 29,160 variables to 121, and the clingo time from 208.9 seconds (baseline) or 161.7 seconds (`la`) to 27 milliseconds, with the same 117 guided choices and zero conflicts, on both backends (the Prolog run measures 79 milliseconds, the difference being the fixed engine startup). The same holds at every size up to $n = 200$, where `la_co` still finishes in 9 milliseconds with 197 choices while every other variant has been timing out for four or five sizes. This measurement completes the diagnosis of Section 4.4: once the lazy heuristic has removed the search cost, the *entire* residual cost of BSP is the ground materialization of the balance check, and a formulation that avoids it removes the last Θ-large term. A comparison between `la_co` and `la` therefore measures the interaction between lazy heuristics and problem modelling, not the isolated effect of the heuristic representation, and it is reported as a modelling experiment: its role is to show what the combination "dynamic heuristic + propagation-friendly model" can reach when neither component pays a combinatorial ground price.

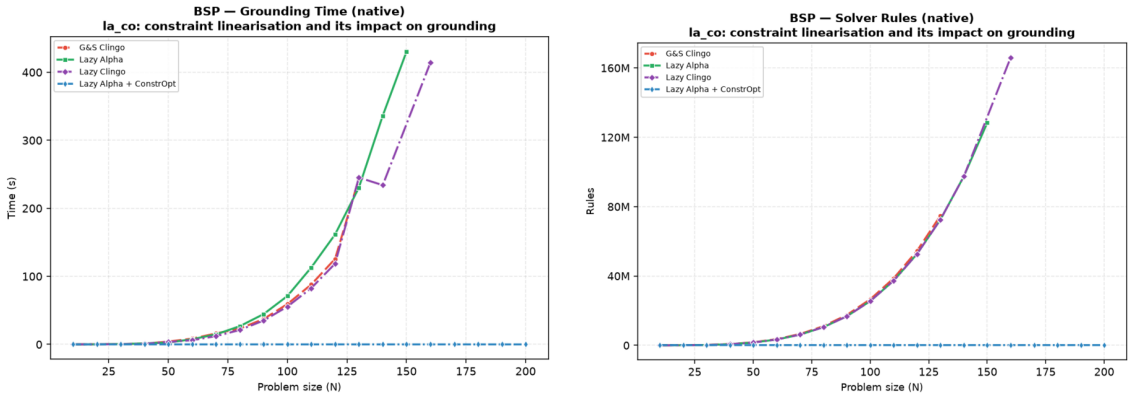

Figure 4.10: BSP, native backend, exploratory comparison of the constraint reformulation: grounding time (left) and propositional rules (right). `gc`, `la`, and `lc` overlap because they share the quadratic base encoding (some 52.8 million rules and one to three minutes of grounding at $n = 120$); `la_co` lies on the horizontal axis of both plots (366 rules, milliseconds of grounding), the visual form of the five-orders-of-magnitude gap discussed in the text.

4.11 PUP: the Grounding Bottleneck at Scale

PUP is the family where the grounding bottleneck, not the search, sets the frontier, and where the lazy representation translates directly into a large saving in time and memory at every common size. Figure 4.11 and Table 4.8 summarize the campaign under the 600-second, 30 GB limits; under this raised budget both Alpha-like variants solve the entire tested range up to $N = 200$, so the frontier that used to separate them has become a cost gap instead.

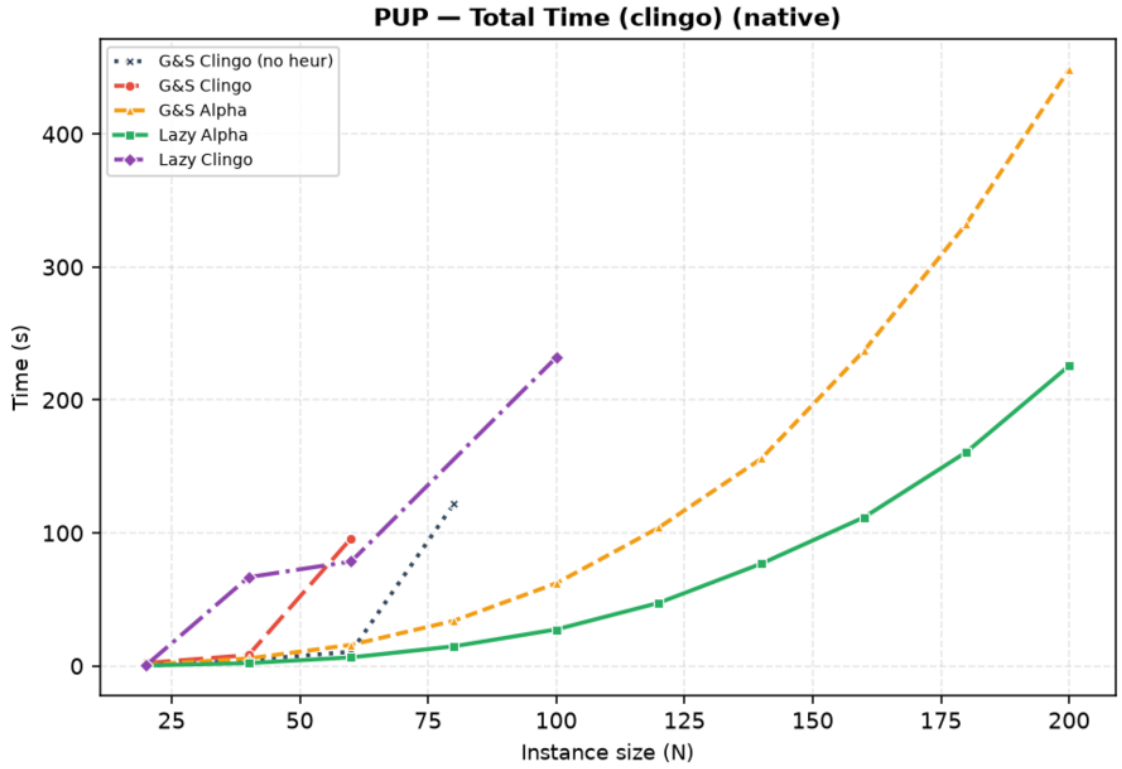

Figure 4.11: PUP, native backend: total clingo time. Each curve ends at the last instance its variant solves.

Variant	max N solved	Total (s)	Grounding (s)	Peak RSS	first failure
`gc_noheur`	80	121.7	10.2	0.7 GB	T@100
`gc`	60	96.0	17.6	1.3 GB	T@80
`ga`	200	448.7	440.1	25.0 GB	—
`la`	200	225.8	220.3	13.0 GB	—
`lc`	100	232.0	27.0	2.1 GB	T@80 (non-mon.)

Table 4.8: PUP frontier, native backend, 600 s / 30 GB per run: largest `double-`N solved and its cost. `ga` and `la` solve the whole tested range and are reported at $N = 200$; the other three still fail inside the range, and are reported at the largest size they solve. Unlike the BSP table, the separations here are between programs of genuinely different size — `ga` grounds twice the memory of `la` at every size — and are therefore an order of magnitude larger than the node spread of Section 4.2.1.

The ordering of the frontiers tells most of the story, and the part that changes with scale tells the rest. The baseline stalls at $N = 80$ by *time*: with no guidance, search explodes first (546,549 choices at $N = 80$, against 272 for `la` at $N = 200$, two and a half times that size). The plain ground heuristic `gc` fares worse still, failing already at $N = 80$ and never recovering: its directives are cheap to ground but do not carry the dynamic aggregate, so they neither prevent the search explosion (926,646 choices before the kill at $N = 80$, against the baseline's 546,549 on the instance it does solve) nor pay for themselves. Both Alpha-like variants clear the

whole tested range under the raised budget, but at markedly different cost. `ga` pays for its static unrolling on every instance: at $N = 120$ it needs 104.0 seconds and 7.0 GB against `la`'s 47.4 seconds and 3.4 GB, a 2.2× time and 2.0× memory gap that persists at every common size, because the unrolled directives grow with the value domains being unrolled, not just with the instance; at $N = 200$ it is 448.7 against 225.8 seconds (1.99×) and 25.0 against 13.0 GB (1.93×). Both variants show the same near-perfect search quality (235 choices and zero conflicts for `ga` against 162 choices and 16 conflicts for `la` at $N = 120$; both semantics-Alpha), so the entire gap is attributable to the heuristic's own grounding, exactly as the mechanism predicts. Two independent facts make this the most robust comparison in the campaign: the gap is a stable factor of two across ten sizes, an order of magnitude above the node spread of Section 4.2.1, and it shows up identically in peak RSS, which that spread does not affect at all. Extrapolating the trend, `ga` is the variant one would expect to hit a memory wall first if the range were pushed further, since it reaches 25.0 of the available 29.3 GiB already at $N = 200$ while `la` is at 13.0; the campaign as run does not observe that wall directly, but the ratio is the same signature that, under the previous 8 GiB budget, made `ga` the first of the two to fail.

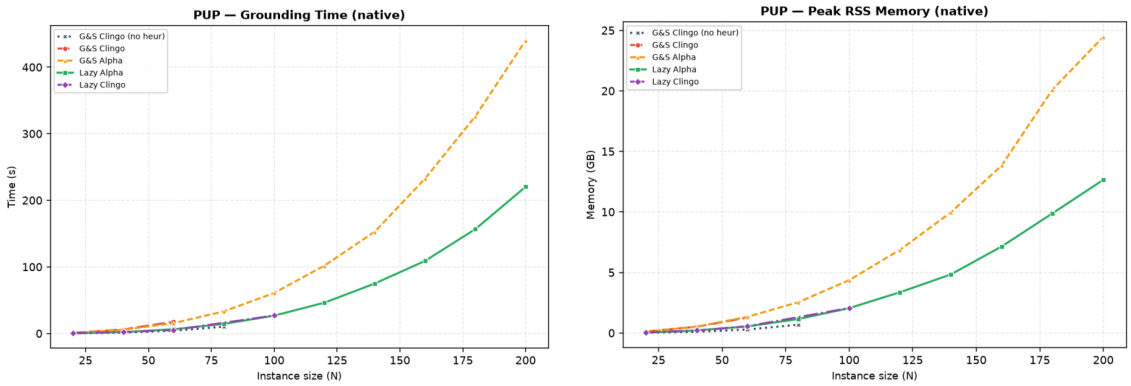

Figure 4.12: PUP, native backend: grounding time and peak memory. `ga` pays the heuristic unrolling (orange) on top of the base growth; `la` (green) pays the base program only, and stays consistently cheaper at every common size.

The clingo-semantics cells complete the picture from below. Native `lc` oscillates around the baseline, solving $N = 100$ (which the baseline misses) but failing at $N = 80$ (which the baseline solves): as on BSP, its gated aggregates never produce candidates (on every instance it solves, both backends report *zero* candidates and a `max_candidates_seen` of zero, over 233 and 36,325 native decide calls). Unlike BSP, however, the trajectory does diverge from the baseline — 36,325 choices against 11,659 at $N = 40$, 174,593 against 3,362,491 at $N = 100$ — which is at first sight a contradiction: a propagator that never returns a candidate cannot change a decision. The resolution is in the ground programs. On BSP the lazy encoding and the baseline instantiate the same program up to the two `__heuristic` facts (52,759,258 rules against 52,759,256), which is why an inert propagator there implies a bit-identical trajectory. On PUP the lazy encoding needs auxiliary predicates to expose the aggregate sources, so it grounds a visibly different instance (216,193 rules and

51,776 variables at $N = 20$, against 161,018 and 5,298 for the baseline): clasp sees a different formula and, on a family this sensitive to it, takes a different trajectory from the first restart. The perturbation is therefore incidental rather than heuristic in origin, it moves in both directions, and it is the honest reading of the non-monotone frontier reported in Table 4.2.

What this same fact buys, however, is the cleanest semantics experiment of the campaign. On PUP `la` and `lc` ground *exactly* the same program — identical rules, atoms, variables and constraints at all ten sizes — and differ only in the `__semantics` token that selects how the propagator reads partial information. Everything that separates them is therefore the semantics and nothing else: at $N = 120$, 162 choices and 47.4 seconds against 57,820 choices and a timeout; at $N = 200$, 272 choices against a timeout. The same comparison on the ground side is impossible to set up, because `ga` and `gc` necessarily differ in their unrolling as well. On the Prolog backend the same inert consultations acquire a price (25,197 calls and 76.3 seconds of synchronization, 72% of the run, already at $N = 40$), and `lc` collapses to $N = 40$: the pathological corner of Section 4.7 reproduced at scale.

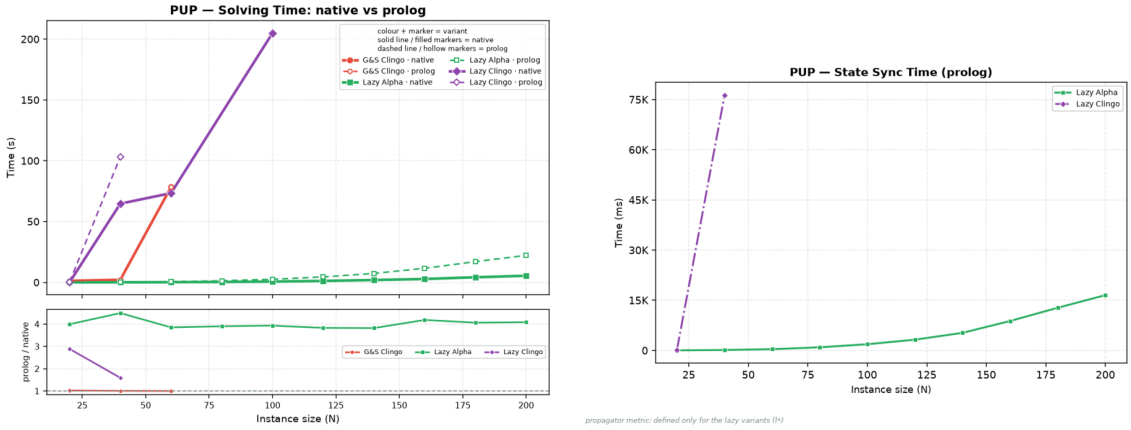

Figure 4.13: PUP: native-versus-Prolog solving comparison (left) and cumulative synchronization cost of the Prolog backend (right). For `la` the synchronization, not the query, is the dominant lazy cost at scale.

For the well-behaved `la`, the cross-backend comparison isolates the pure cost of the general-purpose engine: the guidance is equivalent (both backends solve the whole range, with search statistics within a few percent of each other — 217 against 197 choices at $N = 160$ — the residual difference being the tie-break discussed at the start of this chapter), and the Prolog total grows from 111.8 to 129.0 seconds at $N = 160$ (+15%) and from 225.8 to 248.6 at $N = 200$ (+10%), of which 8.8 and 16.5 seconds respectively are trail synchronization, against 0.8 and 1.7 of query time. At PUP scale the bottleneck of the embedded engine is thus *keeping the state*, not answering the queries — it costs ten times the queries it serves — which points directly at the batched-synchronization optimization discussed in future work. The native backend, whose synchronization is incremental and in-process, spends 0.67 seconds on the same task at $N = 200$.

4.12 HRP: Semantics Without Aggregates

HRP is the negation-heavy benchmark: its five-level reference heuristic [8] contains no aggregates, so the `la`/`lc` comparison isolates the negation semantics alone. At the benchmark sizes the problem is easy for clasp, every variant except `lc` solves every instance in well under a second on the native backend, which makes HRP useless for frontier claims but ideal as a controlled experiment on guidance quality and per-decision cost.

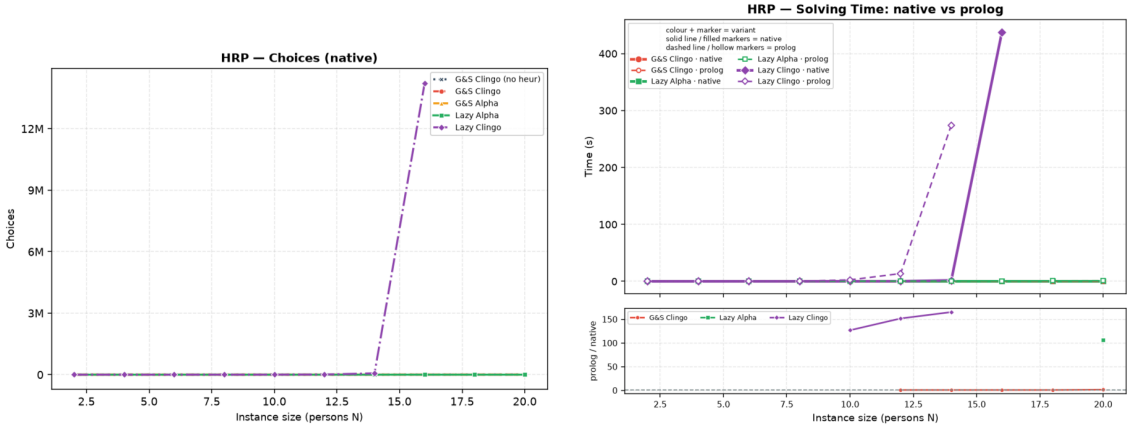

Figure 4.14: HRP: solver choices on the native backend (left) and the native-versus-Prolog solving comparison (right). The only diverging curve is `lc`, in both plots.

The Alpha reading behaves exactly as designed: `la` tracks `ga` decision for decision (86 versus 88 choices at $N = 14$, zero conflicts for both, and the same agreement at every size), confirming on a second problem class that the lazy propagator reproduces the intended Alpha guidance. The clingo reading, by contrast, is *actively toxic* on this heuristic: requiring established falsity on guards such as `not fullCabinet(C)` delays every level of the policy until the corresponding closure information exists, and the surviving low-level modifications steer the solver into a pathological region. Native `lc` needs 66,717 choices and 32,353 conflicts at $N = 14$ (almost eight hundred times the `la` count), and the divergence is super-exponential in N: at $N = 16$ it takes 14,218,384 choices and 8,136,557 conflicts, a factor of two hundred over the previous size, which it still just manages to close in 438.1 of the 600 available seconds; from $N = 18$ it times out. The Prolog twin is one size behind, timing out already from $N = 16$.

Two details of this comparison are worth stating precisely, because they qualify how much the frontier can be made to carry. First, the two backends do *not* reproduce identical trajectories here: at $N = 14$ native `lc` records 66,717 choices and 32,353 conflicts against the Prolog twin's 73,143 and 41,471, and similar few-percent gaps appear at every size. HRP is the family in which the heuristic produces hundreds of equally ranked candidates per decision, so the deterministic tie-break of the two implementations resolves them differently, and once a search this unstable has taken one different decision the counts drift apart. What certifies the collapse as a property of the *semantics* rather than of one engine is therefore not the identity

of the counts but the fact that both backends reach the same pathological order of magnitude, four to five orders above `la`, from encodings that differ only in the semantics token. Second, and for the same reason, the one-size gap between the frontiers is not purely per-decision cost: part of it is the different trajectory. The claim the data supports is the qualitative one — the clingo reading destroys this heuristic on both backends — and it is a strong claim precisely because it survives the change of engine. Where BSP showed the clingo reading as inert, HRP shows it as misleading: the two failure modes of "late" information are both on record.

The Prolog backend adds the cost dimension, and HRP is where it is most visible. The reference rules enumerate every applicable (cabinet, thing) and (room, cabinet) pair, more than a thousand candidates per decision at $N = 20$, so even the well-guided `la` pays 9.7 milliseconds per consultation — the most expensive query in the campaign — and spends 68% of a 1.5-second run inside the propagator, against 0.40 seconds for the native backend, whose compiled candidate scan answers the same query in 10^{-4} milliseconds. The misguided `lc` multiplies 3.6 milliseconds by 73,143 consultations: 248 of its 275 seconds at $N = 14$ are spent inside candidate-rich Prolog queries and 12 more in synchronization, 99.4% of the run, against 1.78 seconds for the native backend on the same instance. That $\times 154$ ratio is the largest in the whole campaign, and it is a measurement of the evaluation engine, not of the heuristic: the search it drives is equally bad on both sides, and only the price of asking differs.

4.13 An External Reference: Alpha

Everything measured so far compares clingo against clingo. It establishes what the lazy representation costs and what the Alpha reading buys *inside* a ground-and-solve system, but it cannot say how far that gets one from the system whose semantics is being reproduced. The runs of Section 3.3.4 answer that question directly, on the two families whose heuristics carry dynamic aggregates. As stated there, only wall-clock time and peak RSS are commensurable across the two systems, and Alpha's memory is inflated by the reserved JVM heap; the internal counters of the two solvers are not compared at any point below, and the frontiers are read as lower bounds within the campaign's limits.

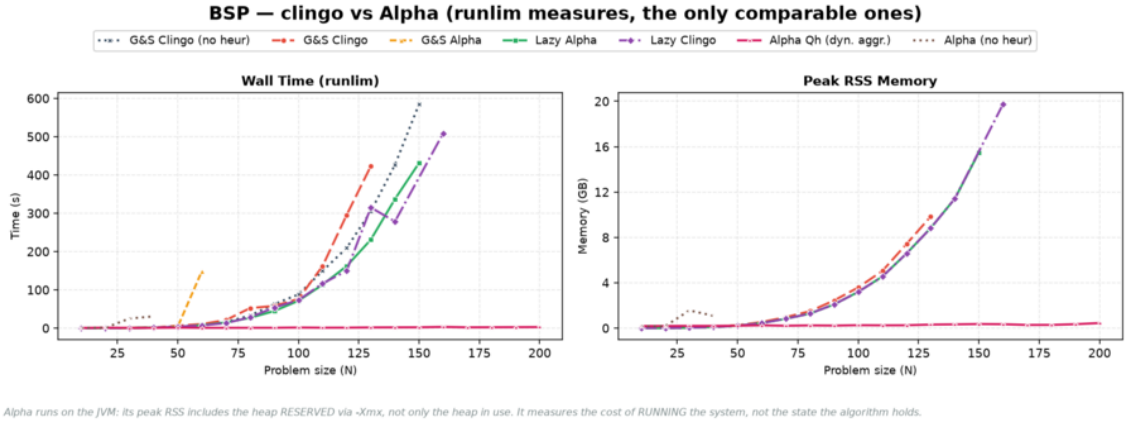

Figure 4.15: BSP: the four clingo variants and the heuristic-free clingo baseline against Alpha with and without its declarative heuristic. Left, wall time; right, peak RSS. The two Alpha curves lie on the horizontal axis of both plots for the whole range; `Alpha (no heur)` instead stops at $n = 40$, earlier than any clingo variant.

On BSP the picture is unambiguous and it is not about the heuristic. Alpha with its heuristic solves all twenty sizes, from 0.6 seconds and 0.18 GB at $n = 10$ to 2.7 seconds and 0.45 GB at $n = 200$, with zero backtracks at every size and a number of guesses equal to n — the same shape of trajectory that `la` produces inside clingo (n choices, zero conflicts), obtained by the system the lazy propagator imitates. Where the two diverge is everything around the search: at $n = 150$, the largest size the clingo lazy variant reaches, `la` needs 431 seconds and 15.5 GB against Alpha's 2.1 seconds and 0.36 GB, and from $n = 160$ every clingo variant except `la_co` is out of the budget while Alpha's cost is still growing linearly and is still under three seconds at $n = 200$.

That gap is exactly the term Section 4.4 isolated and could not remove. `la` already spends 0.04 seconds of solving on a 162-second run: there is no search left to save, and the entire difference is the 52.8 million rules that gringo materialises at $n = 120$ and that Alpha never builds, because it instantiates only what the search touches. The result is therefore not a defeat of the mechanism but a measurement of its declared scope — the propagator acts on the heuristic component and the base program is still grounded — and it is the same conclusion that `la_co` reached from the other direction in Section 4.10, by removing the Θ-large term from the model rather than from the pipeline. Two independent routes, one to $n = 200$ in milliseconds and one to $n = 200$ in seconds, agree that on BSP the residual wall belongs to the domain encoding.

The comparison also settles a question the matrix leaves open, and it settles it in the thesis's favour. Lazy grounding on its own is *not* what carries Alpha: `alpha_noheur` stops at $n = 40$, after 42,058 guesses and 42,021 backtracks, eleven sizes before the ground clingo baseline stops — at a size clingo grounds and solves in under two seconds. Reading the four combinations together — eager grounding without a dynamic heuristic ($n = 150$), eager grounding with one ($n = 150$, but with the search cost removed), lazy grounding without one ($n = 40$), lazy grounding with one ($n = 200$) — shows that the two ingredients are separable and that neither

is sufficient alone. The semantics contribution measured throughout this chapter is thus not an artefact of a weak baseline: it is the same contribution that makes the reference system work, isolated in a setting where grounding is held fixed.

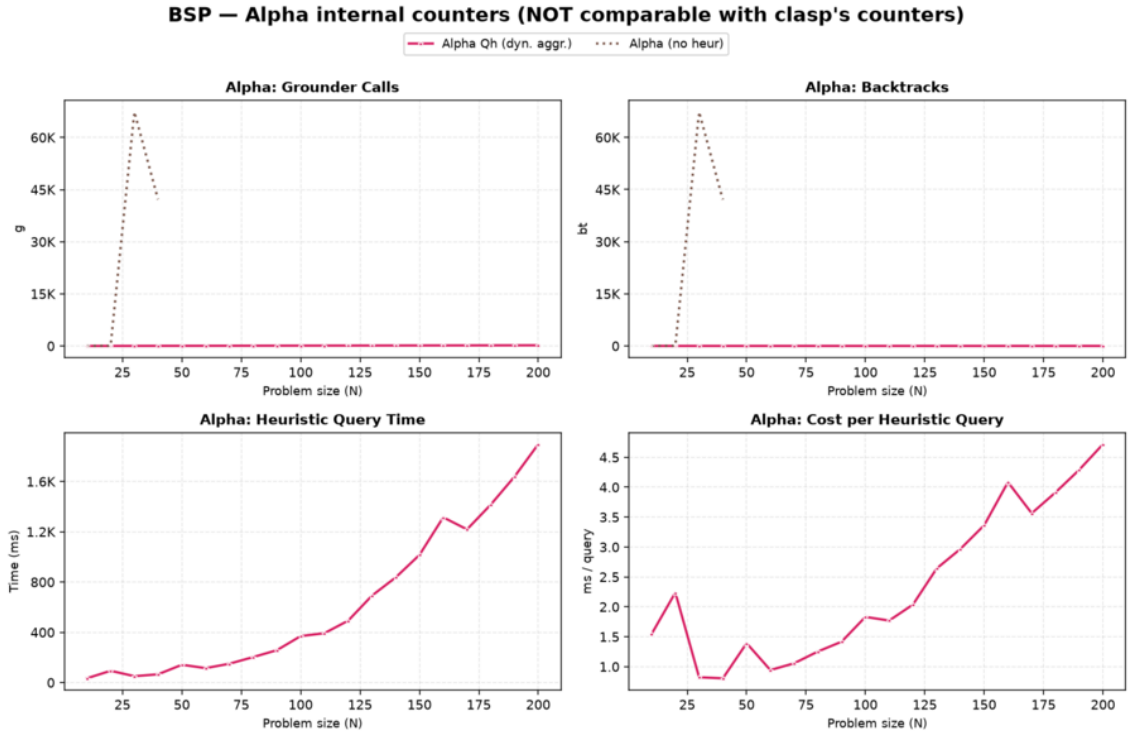

Figure 4.16: BSP, Alpha's own counters, plotted apart from every clingo measure because they are not commensurable with clasp's (Section 3.3.4). Top row: with the heuristic, grounder calls stay equal to n and backtracks at zero over the whole range, while the heuristic-free run spikes to some 67,000 of each at $n = 30$ and stops at $n = 40$. Bottom row: the cost of the heuristic itself, which is a Prolog query in this variant — 1.9 seconds cumulative at $n = 200$, at 0.8 to 4.7 milliseconds per query.

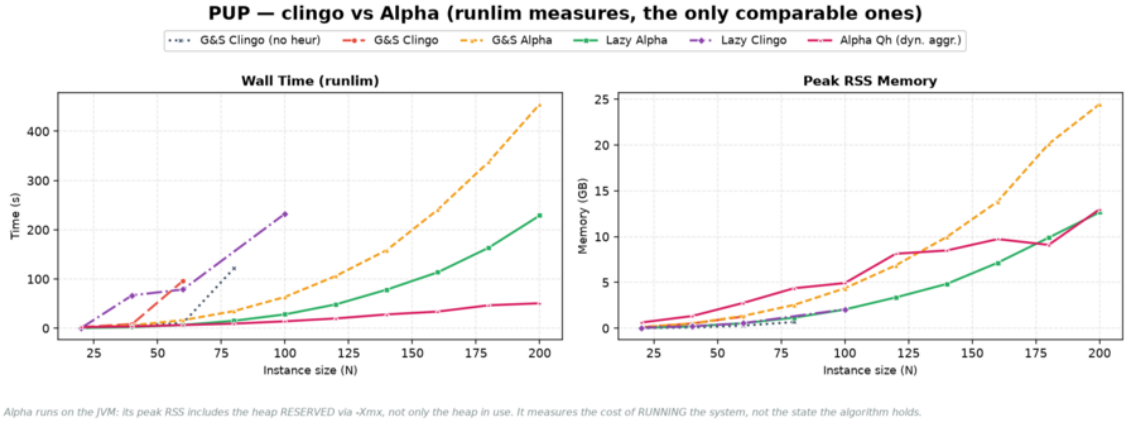

Figure 4.17: PUP: the same comparison. `Alpha (no heur)` does not appear because it solves no instance at all, not even `double-20`. Here the clingo lazy variant stays within a small factor of Alpha in time and matches it in memory, while the ground simulation `ga` is worse than Alpha on both axes.

PUP tells the more interesting half of the story, because there the clingo side is competitive. Alpha solves the whole range with zero backtracks, from 2.2 seconds at $N = 20$ to 50.6 seconds and 13.0 GB at $N = 200$. At that size `la` costs 228.9 seconds and 12.7 GB: a factor of 4.5 in time, and *the same* peak memory — against an Alpha figure that includes its reserved heap and is therefore an over-estimate, which makes the memory comparison a genuine tie at best for the lazy propagator. The ground simulation of the same semantics, `ga`, costs 455.2 seconds and 24.5 GB, that is nine times Alpha's time and nearly twice its memory. The ordering is the one the mechanism predicts: moving the heuristic out of the grounder recovers most of the distance to the native lazy-grounding system, and the part it does not recover is again the base program, since 220 of `la`'s 229 seconds at $N = 200$ are grounding.

Here too the heuristic-free control is decisive, and more sharply than on BSP: `alpha_noheur` solves *nothing*, not even the smallest instance, so on this family the declarative heuristic is the entire reason the reference system is usable at all. The same heuristic, transplanted into clingo and evaluated lazily, gets within a factor of four and a half of it; transplanted and unrolled at grounding time, it ends up slower and hungrier than the system it was copied from.

Family	System	max solved	Wall (s)	Peak RSS
BSP	`alpha_noheur`	40	30.8	1.1 GB
BSP	clingo `gc_noheur`	150	584.9	15.5 GB
BSP	clingo `la`	150	431.3	15.5 GB
BSP	`alpha` (Qh)	200	2.7	0.45 GB
BSP	clingo `la_co`	200	0.03	< 0.1 GB
PUP	`alpha_noheur`	—	—	—
PUP	clingo `gc_noheur`	80	121.8	0.7 GB
PUP	clingo `lc`	100	232.7	2.1 GB
PUP	clingo `ga`	200	455.2	24.5 GB
PUP	clingo `la`	200	228.9	12.7 GB
PUP	`alpha` (Qh)	200	50.6	13.0 GB

Table 4.9: Largest instance solved within the 600 s / 30 GB budget, and its cost, ordered by reach within each family. Wall time and peak RSS are `runlim` measures, the only ones commensurable across the two systems; Alpha's RSS includes the reserved JVM heap. clingo rows are the native backend. A dash marks a system that solved no instance of the family.

One last observation concerns the evaluation engines rather than the systems, and it is possible only because the two designs happen to coincide: Alpha's Qh mode and the Prolog backend of this thesis both answer the heuristic by querying a Prolog engine over an image of the partial assignment. The per-consultation costs are of the same order. At BSP $n = 120$ Alpha issues 242 queries at 2.04 milliseconds each, against 120 consultations at 0.50 milliseconds for the Prolog backend; at PUP $N = 200$, 1,129 queries at 7.12 milliseconds against 249 at 6.74. The two counts measure different events — Alpha queries around its own guesses, the propagator once per clasp decision — so only the order of magnitude is meaningful, but it is enough to say that the synchronisation-based design of Section 3.2.1 does not pay a penalty against the reference implementation of the same idea, and that the three-to-four orders of magnitude separating it from the native backend (Section 4.7) are the price of Prolog evaluation as such, not of this particular way of arranging it.

4.14 Native versus Prolog: Summary

Across the full campaign the two backends put the frontier in the same place in every cell except seven, and those seven fall into two clearly separable groups. Three of them are cells in which the per-decision cost is genuinely the binding constraint, and the Prolog backend is the one that stops earlier: `lc` on PUP ($N = 100 \to 40$), `la_aux` on BSP ($n = 60 \to 30$) and `lc` on HRP ($N = 16 \to 14$). The other four are all on BSP, all on the shared base program, and all in the *opposite* direction — `gc_noheur` ($n = 150 \to 160$), `gc` ($130 \to 150$), `la` ($150 \to 170$) and `lc` ($160 \to 150$) — which is precisely what a node-spread artefact looks like: at the top of the BSP range the grounding of the base program consumes the whole budget, so which side of the 600-second line a run falls on is decided by the node, not by the backend. No claim is made from those four cells. Everywhere the propagator is cheap and the

frontier is not set by grounding, the two backends coincide: on PUP and HRP `la` solves the entire range on both.

For every ground variant the two systems run byte-identical encodings on the same solver, and their choice and conflict counts agree to the unit on all three families, which doubles as an end-to-end sanity check of the whole pipeline. For the lazy variants the native backend is faster per consultation by three to four orders of magnitude, and this is the only cross-backend comparison that is safe to make on time: total times are not, because the grounding component they are dominated by carries the node spread of Section 4.2.1, which is why several BSP cells report a nominally faster Prolog total (for instance 55.0 against 71.2 seconds for `la` at $n = 100$) on a run whose propagator cost 38 milliseconds in total. Where the propagator's share of the run is large, the comparison becomes meaningful again and always favours the native backend, by factors from 1.6 to 154. The Prolog backend remains the more expressive and the more observable one, which is the trade-off it was designed to embody.

4.15 Summary of Results

The full campaign supports eight conclusions. First, the lazy representation reduces the ground heuristic component from $\Theta(n^3)$ directives to two constant facts on BSP, and the saving is confirmed on real memory wherever the heuristic grounding is a significant share of the footprint, culminating in PUP where, under a raised 600-second/30 GB budget that both Alpha-like variants clear over the entire tested range ($N = 20$ to 200), the lazy variant costs half the time and half the peak memory of the ground heuristic simulation at every one of the ten common sizes. Peak memory is the load-bearing measure here, because it is the one the cluster's node heterogeneity cannot perturb.

Second, the total runtime does not become constant, because the base program is still fully grounded, and on BSP this is now the *only* thing that sets the frontier: `gc_noheur`, `la` and `lc` all stop between $n = 150$ and $n = 170$, with the exact size depending on the node rather than on the variant, and what distinguishes them there is not reach but cost, since `la` arrives at the common wall having spent 0.04 seconds of solving against the baseline's 44.8. The lazy mechanism isolates the heuristic from the growth of the instance, no more and no less; where the instance itself is the bottleneck, the correct claim is a thousandfold reduction of the search component, not a better frontier.

Third, the semantics axis is not a detail: the Alpha reading turns the BSP heuristic into a perfect greedy strategy (n choices, zero conflicts, at every size) and reproduces the intended reference policy on HRP decision for decision, while the clingo reading is inert on BSP (bit-identical to the baseline, with a maximum of zero candidates over 78,565 consultations) and actively harmful on HRP (four to five orders of magnitude more choices, on both backends independently). PUP settles the point in the cleanest possible form, because there `la` and `lc` ground literally the same program and differ by one token: 272 choices against a timeout. Representation and semantics must never be conflated.

Fourth, the faithful *ground* simulation of the Alpha reading (`ga`) reproduces the

lazy guidance exactly while its weights remain representable, and stops working once they do not. On BSP it is exact up to $n = 50$, degrades from $n = 51$, and from $n = 70$ is the only variant of the main matrix to fail before the heuristic-free baseline does, by nine sizes, because the reconstructed value of the aggregate no longer fits clingo's weight field. On PUP, where it does fit, it survives the whole tested range but costs twice the time and twice the memory of the lazy variant at every size. Taken together, this makes the lazy propagator not merely cheaper but the only practical carrier of that semantics inside clingo at scale.

Fifth, laziness is a space–time trade-off whose runtime side is now measured on both backends, not merely declared: the overhead factorizes as decide calls times per-consultation cost, the propagator's share of a run ranges from nothing (BSP `la`, 63 milliseconds on a two-minute run) to 99.4% (HRP `lc`), the cost of one consultation differs between the compiled and the interpreted backend by three to four orders of magnitude, and the total is smallest exactly when the heuristic guides well.

Sixth, the three exploratory controls delimit the approach from every side: `ga_weak` shows that dropping the negative guards, without also unrolling the aggregate, is behaviourally inert while still paying the full $\Theta(n^3)$ grounding, `la_aux` shows that materializing the heuristic body forfeits the benefit entirely and on the Prolog side turns synchronization into 99% of the run, and `la_co` shows that pairing the lazy heuristic with a propagation-friendly model eliminates the residual ground cost and solves the entire range, $n = 200$ included, in milliseconds.

Seventh, the two backends agree on every scientific conclusion and on every frontier except the three cells where the per-decision cost is itself the binding constraint, so the choice between them is a genuine engineering trade-off rather than a correctness question.

Eighth, the external comparison with Alpha places all of the above on an absolute scale and separates its two ingredients. The lazy propagator reproduces the reference system's guidance — on BSP the same n-decision, backtrack-free trajectory — and on PUP comes within a factor of 4.5 in time and matches it in peak memory, while the ground simulation of the same semantics is nine times slower and twice as hungry as the reference; on BSP the residual gap is two orders of magnitude and is entirely the grounding of the base program, which is the mechanism's declared scope limit and the same term that `la_co` removes from the other side. Crucially, Alpha *without* its heuristic stops at $n = 40$ on BSP and solves nothing at all on PUP, so lazy grounding and dynamic heuristics are separable and neither is sufficient alone: the semantics effect measured throughout this chapter is the same one that makes the reference system work.

Chapter 5

Discussion and Conclusions

5.1 Discussion of the Results

The work shows that an important part of the cost of declarative dynamic heuristics can be moved from grounding to search. On BSP, native `#heuristic` directives grow as $\Theta(n^3)$ and exceed one million ground objects at $n = 100$; the lazy variants keep two facts, and the difference is visible in the propositional instance size, in the peak memory, and in the timeout profile. The mechanism generalizes, and the full campaign measures the generalization: on PUP, whose heuristics multiply four unrolled value domains per ground pair, the ground Alpha simulation and the lazy variant both clear the entire tested range under a raised 600-second/30 GB budget, but the ground simulation costs twice the time and twice the memory of the lazy variant at every one of the ten common sizes, because the unrolled directives keep growing with the value domains being unrolled; both dominate the unguided baseline, which stalls by timeout at less than half their reach. Where the heuristic grounding is the binding constraint, the lazy representation converts directly into a large cost advantage over the best ground heuristic, and into solved instances relative to the unguided baseline; where the base program dominates, as at the top of the BSP range, it converts into an unchanged frontier reached with the search component reduced by three orders of magnitude, and the residual wall belongs to the domain encoding, not to the heuristic. It is worth being explicit that the BSP frontier is genuinely unchanged and not improved: the lazy variant, the clingo-like one and the heuristic-free baseline all stop within two sizes of each other, they do not even keep the same order between the two backends, and the differences among them there fall inside the node-to-node spread of the cluster's grounding times. On that family the honest claim is about cost, not about reach, and the campaign's design — structural counters alongside wall-clock times — is what makes the distinction visible instead of letting a favourable node stand in for a result.

The external comparison of Section 4.13 puts a number on both halves of that sentence, and it is the most informative single experiment of the campaign because it is the only one whose yardstick is not clingo. Against Alpha in its dynamic-aggregate variant — the system whose semantics this thesis reproduces, running the authors' own encodings — the lazy propagator delivers the guidance and not the grounding. On BSP it reproduces the reference trajectory exactly, n decisions and

no backtracking, and then loses two orders of magnitude in wall time to a system that never builds the 52.8 million rules of the base program. On PUP, where the base program is proportionally less dominant, it lands within a factor of 4.5 in time and level in peak memory, while the ground simulation of the very same heuristic is nine times slower and twice as hungry as the reference. Read as a whole the ordering is `ga` $<$ `la` $<$ Alpha, with the lazy representation recovering most of the distance and grounding accounting for all of the remainder — which is precisely the claim the thesis makes, now measured against an external reference rather than asserted.

The same experiment also protects the semantic claim from an obvious objection, namely that the Alpha reading only looks good because it is being compared with clingo baselines. Alpha *without* its declarative heuristic stops at $n = 40$ on BSP, eleven sizes before ground clingo without a heuristic, and solves no PUP instance at all. Lazy grounding and dynamic heuristics are therefore separable contributions and neither is sufficient on its own: the four combinations of (eager, lazy) grounding and (absent, dynamic) heuristic land at $n = 150$, $n = 150$, $n = 40$ and $n = 200$ respectively. The effect this thesis isolates inside clingo is the same effect that makes the reference system usable.

This result should not be read as a replacement for general lazy grounding. clingo still grounds the base program; the propagator acts only on the heuristic component, and Section 4.13 measures exactly what that leaves on the table. Nor should it be read as a claim that laziness is free: the runtime work is real, it is paid at every decision, and the per-decision metrics exist precisely to keep that cost on the record. The honest formulation is a space–time trade-off with a crossover: below it, ground-and-solve remains competitive, because the explosion is manageable and the lazy machinery (in the Prolog case, an entire in-process logic-programming engine) carries a fixed overhead; beyond it, the constant-size heuristic component is the only representation that survives.

The second central finding is semantic, and it is a finding about *meaning*, not performance. The same declarative heuristic changes behaviour dramatically depending on how partial information is read. Under the Alpha-like reading, the BSP heuristic is a perfect greedy strategy (n choices, zero conflicts) and the HRP policy reproduces its intended level structure decision for decision. Under the clingo-like reading, the very same weights and bodies degenerate in two distinct ways that the campaign now documents separately: on BSP the guards never open at decision time, the propagator reports zero applicable candidates per call, and the search is bit-identical to the baseline (inertness); on HRP the surviving low-level modifications actively steer the solver into a region almost eight hundred times larger at $N = 14$ and five orders of magnitude larger at $N = 16$, with both backends independently reaching the same pathological order of magnitude of choices even though their exact counts differ (misguidance). Neither reading is "wrong": they answer different questions, and the four-variant matrix exists to keep those questions separate. A comparison that silently mixed the axes, for example by measuring `la` against `gc` and attributing the difference to laziness, would be methodologically invalid; the thesis takes some care never to do so. The campaign adds a corollary about the ground side of the matrix. The faithful ground simulation of the Alpha reading, `ga`, reproduces the lazy guidance exactly while its weights remain representable, and stops

working when they do not: on BSP that happens at $n = 51$, and from $n = 70$ the variant no longer finishes at all, making it the only variant of the main matrix that times out before the baseline does on BSP. The limit is not the cost of the unrolling, which on that family adds half a second of grounding and thirty-six megabytes, but clingo's weight field, into which the reconstructed value of the aggregate has to fit. Inside a ground-and-solve pipeline the lazy propagator is therefore not merely the cheaper carrier of the Alpha semantics; beyond a certain instance size it is the only one that can carry it at all, because it ranks its targets internally and never has to encode the aggregate as a weight.

The third finding concerns the translation layer between the two heuristic languages. Alpha's weight–priority pairs are global levels; clingo's priorities arbitrate on a single atom. Every encoding that expresses an ordering between different atom families must therefore flatten its levels into weights wherever clingo's reading applies: in the ground variants, because clasp is the consumer, and in the lazy clingo variants, because the propagator deliberately reproduces clingo's behaviour there. This is easy to state and easy to get wrong: an early revision of the lazy-clingo PUP and HRP encodings carried Alpha-style priorities that the clingo semantics does not rank globally, producing an unintended decision order that inverted the designed one (sensors before zones; cabinet-filling before reuse), and diverging from the Prolog twin encodings of the same variants. The final audit caught and corrected the discrepancy by applying the same flattening used in the ground encodings. The episode is instructive: the interaction between priority semantics and evaluation backend is exactly the kind of silent, non-crashing behavioural divergence that a benchmark pipeline cannot detect from exit codes, and it motivates the equivalence and wiring guards that the suite now includes.

The lesson was then turned into a structural decision rather than a matter of vigilance. As long as the priority field is used to carry levels in *some* encodings, its meaning depends on the pair (semantics, backend): a global level under one reading, a purely local arbiter under another, and a global level again in the Prolog backend, whose selection rule sorts by (priority, weight) regardless of the semantics selector. Three readings of one syntactic field is a standing invitation to the very bug just described. Since flattening is order-preserving, the suite removes the ambiguity by construction: the levels always live in the weight, in every variant and in both backends. The native heuristic language went one step further and dropped the priority construct altogether, so that a directive written for it cannot express a level except through its weight, and the parser rejects the field rather than silently reinterpreting it. Two properties follow. First, the encodings of a pair differ by exactly one token, so the axis under study is the only thing that varies. Second, the change is behaviour-preserving with respect to the earlier revision: on every benchmark the flattened weights reproduce the previous total order exactly, because on each instance the level multiplier strictly dominates the intra-level weights, so the experimental results reported here are unaffected.

A related engineering lesson comes from the silent-fallback failure mode of the Prolog backend. A single missing environment variable degrades every lazy Prolog run to the no-heuristic baseline without any error, since the fallback evaluator simply fails on the Prolog-style rule bodies. The pipeline defends itself with posi-

tive evidence of activity (`decide_calls > 0`), cross-backend agreement checks, and a wrapper that forces the variable; the general principle, that a heuristic mechanism must prove it ran rather than merely not fail, seems worth stating explicitly.

Finally, the auxiliary-form experiment delimits the approach from within. `la_aux` shows that laziness only pays if the *entire* dynamic part of the heuristic, aggregate included, stays out of the grounder: materializing the heuristic body into an auxiliary relation reintroduces the very product the mechanism was built to avoid.

A fourth lesson concerns the interface between the heuristic language and the solver that consumes it. clingo represents the weight of a `#heuristic` modification in a bounded field and saturates anything larger, so a weight is not an arbitrary integer but a resource with a ceiling. Any encoding that packs information into the weight, whether an Alpha level through the flattening rule of Section 2.1.2 or the value of an aggregate through the unrolling of `ga`, is spending that resource, and the two uses compete. Nothing warns the modeller when the ceiling is reached: the program still grounds, the run still terminates, the answer sets are still correct, and only the search degrades. This is the same class of silent failure as the priority episode and the Prolog fallback described above, and it argues for the same remedy, namely that a heuristic layer should check its own preconditions rather than trust that they hold. The lazy propagator sidesteps the ceiling by construction, since it ranks targets internally and hands the solver a decision rather than a weight, and that is a design advantage worth stating separately from the grounding one.

5.2 Limitations

The current implementation has explicit technical limitations. The native backend indexes targets and bodies through numeric tuples only; non-numeric arguments are not supported. The arithmetic language covers addition, subtraction, and multiplication; the available dynamic aggregates are sum, count, min, and max, over a single source predicate, with joins delegated to precompiled auxiliary predicates in the encoding. The modifiers `init` and `factor` are recognized but deliberately rejected: in native clingo they modify solver-internal scores that the `decide` interface cannot reach, and approximating them as `level` would silently change their meaning. The propagator is registered sequentially and has no synchronization layer for parallel solving. The tie-break between targets of equal rank is deterministic but does not reproduce clingo's internal scores.

On the evaluation side, the campaign now covers all three families on both backends with the corrected encodings, and both backends are phase-timed, but its scope must be stated precisely. The most important limitation is one of measurement rather than of design: jobs are dispatched by SLURM across heterogeneous nodes, and the grounding time of one and the same program varies between them by a median of 28% and by up to 46%. Every time-based claim in this thesis is therefore qualified by that spread, and only the claims whose margin is an order of magnitude larger — the factor of two between `ga` and `la` on PUP, the two orders of magnitude of `lc` on HRP, the three orders of magnitude on the BSP solving component — are load-bearing. The structural counters, which are exact, carry the rest. A campaign with repeated runs per cell, or pinned to a homogeneous partition,

would let the BSP frontier itself be claimed rather than only the cost behind it; this was not affordable within the cluster budget and remains the single most valuable extension of the experimental protocol.

The HRP instances remain easy for clasp at every benchmarked size, so HRP supports conclusions about guidance quality and per-decision cost, not about frontiers. The PUP frontier claims rest on one instance per size (the `double` family): the non-monotone behaviour observed for `gc` and `lc` is a reminder that neighbouring sizes differ in hardness, and per-size variance is not measured. The two backends also do not reproduce identical search trajectories on PUP and HRP, where the heuristic ranking admits ties that the C++ and Prolog tie-breaks resolve differently; the divergence is a few percent on `la` and larger on the unstable `lc` searches, which is why the cross-backend comparisons in those families are read qualitatively rather than as controlled measurements of the engine alone. The memory metric is end-to-end process RSS, which is the honest but blunt instrument discussed in Chapters 3 and 4: it includes the Prolog engine for the lazy Prolog variants and cannot separate grounding memory from search memory. The auxiliary clingo evaluator of the Prolog build re-grounds a program per decision and is a functional reference, not a performance-comparable backend. The external comparison with Alpha inherits limitations of its own: it covers BSP and PUP but not HRP, whose reference heuristic contains no dynamic aggregates and would therefore measure a different thing; the two solvers' internal counters are not commensurable, so the comparison rests on the two `runlim` measures alone; and Alpha's peak RSS includes the JVM heap reserved through `-Xmx` rather than the heap in use, which makes its memory figures an upper bound and the memory comparison on PUP a tie read conservatively rather than a measured equality. Finally, the whitelist of static predicates and the inference of the program constant n in the Prolog backend are benchmark-specific conveniences that a production version should replace with a declarative interface.

5.3 Implications

The main implication is that declarative heuristics can be treated as an intermediate layer between modelling and solving. One does not have to choose between fully grounded `#heuristic` directives and procedural heuristics hard-coded in the solver: a compact declarative surface, evaluated at runtime with access to the partial assignment, is a workable third point in the design space, and it can host more than one evaluation semantics behind one syntax. The two backends demonstrate two implementation strategies for that layer, a compiled, incremental one and an embedded general-purpose engine, with the expected trade-off between per-decision efficiency and expressive freedom of the rule bodies.

From an engineering perspective, the work also shows that clingo's propagator and heuristic interfaces are flexible enough to prototype non-trivial guidance mechanisms without touching the solver core: watched literals, incremental aggregate states, per-target modifier resolution, and a global decision ranking are all expressible against the public API of `Clingo::Heuristic`.

5.4 Future Work

Several extensions follow naturally. The mechanism should become configurable rather than always-on, and the expression language deserves integer division, modulo, and comparisons, together with a principled treatment of non-numeric symbols through a stable internal mapping. Aggregates with in-template joins would remove the need for precompiled auxiliary predicates. On the Prolog side, the synchronization protocol could move from per-literal goals to batched updates, and the backend could expose its per-decision statistics through clingo's statistics interface instead of stderr parsing. A parallel-solving synchronization layer and a tie-break policy that cooperates with the solver's own scores are natural solver-side improvements.

On the evaluation side, the PUP synchronization numbers make the batched-update optimization concrete and measurable: at `double-160` the trail mirroring costs twelve times the queries it serves (8.8 seconds against 0.8), and at `double-200` ten times (16.5 against 1.7), while the native backend performs the same task in 0.67 seconds; a protocol that coalesces retract–assert pairs per propagation batch therefore has both a quantified target to beat and an existence proof that the target is reachable. Beyond that, configuration, scheduling, and packing domains are natural candidates for broadening the evaluation, precisely because their useful heuristics are dynamic; PUP would additionally benefit from multiple instances per size (the `doubleV` family) to put variance bars behind the frontier claims. A further direction is a verified converter from annotated `#heuristic` directives to lazy facts, with the flattening rule of Section 2.1.2 applied mechanically rather than by hand; the audit episode discussed above is a concrete argument for automating that step.

5.5 Conclusions

This thesis presented a lazy evaluation mechanism for declarative domain-specific heuristics in clingo, instantiated in two backends over a common propagator design: a native C++ backend interpreting compact `__heuristic` facts with incremental dynamic aggregates, and an in-process SWI-Prolog backend evaluating heuristic rules as logic programs against a synchronized image of the solver trail. The mechanism supports two explicit readings of partial information, the clingo-like and the Alpha-like semantics, and reproduces clingo's weight, priority, and modifier behaviour where that reading applies, including the systematic flattening of Alpha's global priority levels into clingo weights.

The full campaign on BSP, PUP, and HRP under uniform limits (600 seconds, 30 GB) shows a reduction of the ground heuristic component from $\Theta(n^3)$ directives, more than one million at $n = 100$, to two constant facts, with the corresponding saving in real memory wherever the heuristic grounding matters; on PUP the lazy variant reaches more than twice the solvable frontier of the unguided baseline and costs half the time and half the memory of the best ground heuristic at every size both solve, while on BSP, where the base program owns the frontier, it removes the search cost outright — n choices and zero conflicts at every size, against tens of thousands for every alternative — and reaches the same wall as the baseline having spent 0.04 seconds of solving where the baseline spends 45. The campaign also

quantifies the price, on both backends: a per-decision runtime cost that ground-and-solve does not pay, which factorizes into consultation count times consultation cost, ranges from nothing to 99% of a run, differs between the compiled and the interpreted evaluator by three to four orders of magnitude per consultation, vanishes when the heuristic guides well, and dominates when it does not. The suite validates that all variants and both backends preserve the answer sets and guards against silent misconfiguration. Measured against Alpha itself, running the authors' encodings, the lazy propagator reproduces the reference guidance and closes most of the distance to a native lazy-grounding solver — within a factor of 4.5 in time and level in memory on PUP, where a ground simulation of the same heuristic is instead nine times slower than the reference — with the residual gap consisting entirely of the base program that clingo still grounds.

The final result is a working, documented, and measurable prototype together with a methodology: two orthogonal axes that keep representation and semantics separate, a flattening rule that makes Alpha-style encodings meaningful under clingo's reading of priorities, and an experimental discipline that states what each number measures. It does not solve the grounding bottleneck in general, but it provides a concrete, verifiable path for bringing dynamic declarative heuristics into clingo without always paying the cost of their full ground materialization.

Bibliography

[1] Michael Gelfond and Vladimir Lifschitz. "The Stable Model Semantics for Logic Programming". In: *Proceedings of the Fifth International Conference and Symposium on Logic Programming*. MIT Press, 1988, pp. 1070–1080.

[2] Torsten Schaub. *Answer Set Solving in Practice*. Potassco slide package, University of Potsdam. Version 1.22.0, 4 October 2024. 2024. URL: `https://teaching.potassco.org`.

[3] Martin Gebser et al. *Potassco Guide*. Second edition, version 2.2.0. University of Potsdam. 2019. URL: `https://potassco.org`.

[4] Francesco Calimeri et al. "ASP-Core-2 Input Language Format". In: *Theory and Practice of Logic Programming* 20.2 (2020), pp. 294–309.

[5] Martin Gebser et al. "Domain-Specific Heuristics in Answer Set Programming". In: *Proceedings of the Twenty-Seventh AAAI Conference on Artificial Intelligence*. 2013, pp. 350–356.

[6] Carmine Dodaro et al. "Combining Answer Set Programming and Domain Heuristics for Solving Hard Industrial Problems". In: *Theory and Practice of Logic Programming* 16.5-6 (2016), pp. 653–669.

[7] Antonius Weinzierl. "Blending Lazy-Grounding and CDNL Search for Answer-Set Solving". In: *Logic Programming and Nonmonotonic Reasoning (LPNMR 2017)*. Vol. 10377. Lecture Notes in Computer Science. Springer, 2017, pp. 191–204.

[8] Richard Comploi-Taupe et al. "Domain-Specific Heuristics in Answer Set Programming: A Declarative Non-Monotonic Approach". In: *Journal of Artificial Intelligence Research* 76 (2023), pp. 59–114.

[9] Richard Comploi-Taupe, Gerhard Friedrich, and Tilman Niestroj. "Dynamic Aggregates in Expressive ASP Heuristics for Configuration Problems". In: *Proceedings of the 25th International Workshop on Configuration (ConfWS 2023)*. Vol. 3509. CEUR Workshop Proceedings. CEUR-WS.org, 2023.

[10] Martin Gebser et al. "Multi-shot ASP solving with clingo". In: *Theory and Practice of Logic Programming* 19.1 (2019), pp. 27–82.

[11] Martin Gebser et al. "clasp: A Conflict-Driven Answer Set Solver". In: *Logic Programming and Nonmonotonic Reasoning (LPNMR 2007)*. Vol. 4483. Lecture Notes in Computer Science. Springer, 2007, pp. 260–265.

[12] Matthew W. Moskewicz et al. "Chaff: Engineering an Efficient SAT Solver". In: *Proceedings of the 38th Design Automation Conference*. 2001.

[13] Jan Wielemaker et al. “SWI-Prolog”. In: *Theory and Practice of Logic Programming* 12.1-2 (2012), pp. 67–96.

[14] Markus Aschinger et al. “Optimization Methods for the Partner Units Problem”. In: *Integration of AI and OR Techniques in Constraint Programming (CPAIOR 2011)*. Vol. 6697. Lecture Notes in Computer Science. Springer, 2011, pp. 4–19.

[15] Gerhard Friedrich et al. “(Re)configuration using Answer Set Programming”. In: *Proceedings of the IJCAI 2011 Workshop on Configuration*. Vol. 755. CEUR Workshop Proceedings. CEUR-WS.org, 2011, pp. 17–24.

[16] Armin Biere. *runlim: a tool for sampling time and memory usage of a process*. `https://github.com/arminbiere/runlim`. Accessed 2026.

[17] Armin Biere and Andreas Fröhlich. “Evaluating CDCL Variable Scoring Schemes”. In: *Theory and Applications of Satisfiability Testing (SAT 2015)*. Vol. 9340. Lecture Notes in Computer Science. Springer, 2015, pp. 405–422.

[18] Carmine Dodaro and Francesco Ricca. “The External Interface for Extending WASP”. In: *Theory and Practice of Logic Programming* 20.2 (2020), pp. 225–248.

Acknowledgements

I would like to thank Prof. Carmine Dodaro for his guidance at the University of Calabria, and Prof. Torsten Schaub and Dr. Javier Romero for their supervision and support during my Erasmus period at the University of Potsdam. Their feedback helped shape both the modelling choices and the implementation decisions behind this work.

I am also grateful to the ASP and Potassco communities. Tools such as clingo make it possible to experiment with advanced research ideas while relying on a solid, well documented, and open foundation.

Finally, I thank my family and the people who stayed close to me throughout this thesis work, for their constant support and patience during the many long debugging sessions.